

UNIVERSIDAD TECNOLÓGICA NACIONAL
Tecnatura Universitaria en Programación
Trabajo Final Integrador

E-commerce con reserva por WhatsApp

Caso de aplicación: Librería Mi Sueño

Arquitectura técnica, fundamentos de decisión y análisis de viabilidad

Autores:

- Franco Ghilardi
- Agustin Chiaravalloti

Tutor: Juan Ignacio Schiavonni

Grupo: 208

Fecha: 27 de Septiembre de 2026

Índice

1. Identificación del problema

- 1.1 Contexto
- 1.2 Problemas identificados
- 1.3 Objetivos del proyecto
 - 1.3.1 Objetivo general
 - 1.3.2 Objetivos específicos

2. Requerimientos identificados

- 2.1 Requerimientos funcionales
- 2.2 Requerimientos no funcionales

3. Propuesta de solución

- 3.1 Visión general
- 3.2 Alcance de la primera implementación (MVP)
- 3.3 Mejora futura (fuera del alcance del MVP)
- 3.4 Relación entre el MVP y la mejora futura

4. Definición del stack tecnológico

- 4.1 Arquitectura de referencia

5. Toma de decisiones de stack fundamentadas

- 5.1 Resumen de decisiones
- 5.2 Frontend: Next.js en Vercel
- 5.3 Backend: NestJS como servicio independiente
- 5.4 Hosting del backend: Render
- 5.5 ORM: Drizzle en lugar de Prisma
- 5.6 Base de datos y almacenamiento: Supabase
- 5.7 Autenticación: JWT propio y refresh token en cookie
- 5.8 Contraseñas: Argon2id
- 5.9 Envío de mails: Resend

6. Análisis de viabilidad

- 6.1 Viabilidad técnica
- 6.2 Riesgos y mitigaciones
- 6.3 Conclusión

7. Bases de Datos

- 7.1 Visión general del modelo
- 7.2 Diagrama entidad-relación
- 7.3 Relaciones entre tablas
- 7.4 Diccionario de datos
- 7.5 Reglas de negocio garantizadas por la base
- 7.6 Interacción entre tablas en las operaciones principales
- 7.7 Decisiones de diseño fundamentadas
- 7.8 Implementación, índices y migraciones
- 7.9 Evolución prevista: pedidos y pagos

8. Backend

- 8.1 Arquitectura en capas
- 8.2 Módulos del sistema
- 8.3 Núcleo compartido
- 8.4 Módulo identity
- 8.5 Módulo catalog
- 8.6 Alta de producto y procesamiento de imágenes
- 8.7 Seguridad y manejo de la sesión
- 8.8 API REST
- 8.9 Persistencia con Drizzle
- 8.10 Validación de las reglas de negocio en capas
- 8.11 Decisiones de diseño fundamentadas

9. Frontend

- 9.1 Criterios de arquitectura
- 9.2 Mapa de rutas
- 9.3 Módulos del sistema
- 9.4 Módulos transversales
- 9.5 Módulos de la tienda pública
- 9.6 Módulos del panel de administración
- 9.7 Pantallas y módulos
- 9.8 Estructura de carpetas
- 9.9 Integración con el backend
- 9.10 Decisiones de diseño fundamentadas

Índice de figuras

Figura 1. Arquitectura de referencia del sistema

Figura 2. Diagrama entidad-relación del modelo de datos

Figura 3. Arquitectura en capas del backend

Figura 4. Módulos del backend y sus componentes principales

Figura 5. Alta de producto y procesamiento de imágenes

Figura 6. Módulos del frontend y sus dependencias

Figura 7. Estructura de carpetas del frontend

Índice de tablas

Tabla 1. Registro de entregas del Trabajo Final Integrador
Tabla 2. Requerimientos funcionales del sistema
Tabla 3. Requerimientos no funcionales del sistema
Tabla 4. Stack tecnológico por capa
Tabla 5. Resumen de las decisiones de stack
Tabla 6. Comparación entre Drizzle y Prisma
Tabla 7. Riesgos del proyecto y sus mitigaciones
Tabla 8. Tablas del modelo de datos
Tabla 9. Relaciones entre las tablas del modelo
Tabla 10. Diccionario de datos de la tabla users: cuentas del panel de administración
Tabla 11. Diccionario de datos de la tabla sessions: sesiones iniciadas a partir de un refresh token
Tabla 12. Diccionario de datos de la tabla categories: categorías y subcategorías del catálogo
Tabla 13. Diccionario de datos de la tabla products: entidad central del catálogo
Tabla 14. Diccionario de datos de la tabla product_categories: asignación de categorías a cada producto
Tabla 15. Diccionario de datos de la tabla related_products: productos similares de cada producto
Tabla 16. Diccionario de datos de la tabla product_images: imágenes de cada producto y su procesamiento
Tabla 17. Reglas de negocio garantizadas por la base de datos
Tabla 18. Decisiones de diseño del modelo de datos
Tabla 19. Capas del backend y sus responsabilidades
Tabla 20. Módulos del backend
Tabla 21. Correspondencia entre errores y códigos de respuesta HTTP
Tabla 22. Clases del dominio del módulo identity
Tabla 23. Casos de uso del módulo identity
Tabla 24. Implementaciones de infraestructura del módulo identity
Tabla 25. Clases del dominio del módulo catalog
Tabla 26. Casos de uso del módulo catalog
Tabla 27. Infraestructura y presentación del módulo catalog
Tabla 28. Configuración de la cookie del refresh token
Tabla 29. Límites de solicitudes por endpoint
Tabla 30. Endpoints de autenticación
Tabla 31. Endpoints de usuarios
Tabla 32. Endpoints de categorías
Tabla 33. Endpoints de productos
Tabla 34. Endpoints de imágenes de producto y búsqueda
Tabla 35. Validación de las reglas de negocio por capa
Tabla 36. Decisiones de diseño del backend
Tabla 37. Capas dentro de cada módulo del frontend
Tabla 38. Librerías del frontend
Tabla 39. Mapa de rutas del frontend
Tabla 40. Módulos del frontend

Tabla 41. Componentes del módulo core

Tabla 42. Mensajes de WhatsApp generados por el sistema

Tabla 43. Componentes del módulo seo

Tabla 44. Componentes del módulo site-layout

Tabla 45. Componentes del módulo catalog

Tabla 46. Guards de ruta del panel

Tabla 47. Pantallas y acciones del módulo auth

Tabla 48. Componentes compartidos del panel

Tabla 49. Campos del formulario de producto

Tabla 50. Pantallas del frontend y módulos que intervienen

Tabla 51. Cobertura de los requerimientos funcionales en el frontend

Tabla 52. Endpoints de la API utilizados por cada módulo del frontend

Tabla 53. Decisiones de diseño del frontend

Registro de entregas

Este Trabajo Final Integrador se desarrolla de forma incremental. Cada entrega agrega o revisa secciones del documento; las anteriores permanecen como base de las siguientes salvo que se indique explícitamente una revisión. La Tabla 1 detalla el contenido de cada entrega.

Tabla 1. Registro de entregas del Trabajo Final Integrador

Entrega	Contenido
1	Identificación del problema, propuesta de solución (MVP y mejora futura), definición y justificación del stack tecnológico, análisis de viabilidad inicial. Revisada en septiembre de 2026 para alinearse con el modelo de datos y el diseño del backend: baja lógica con reactivación, rol vendedor activo en el MVP, envío de mails, modelo de datos sin pedidos ni pagos, refresh token en cookie con dominio compartido, Argon2id y API versionada.
2	Modelo de datos: diagrama entidad-relación, diccionario de datos, relaciones entre tablas, reglas de negocio garantizadas por la base, decisiones de diseño y evolución prevista hacia pedidos y pagos. Diseño del backend: arquitectura en capas, módulos identity y catalog, flujos principales, seguridad y manejo de sesión, endpoints de la API REST y decisiones de diseño. Diseño del frontend: criterios de arquitectura, mapa de rutas, módulos de la tienda pública y del panel, pantallas, estructura de carpetas, integración con la API y decisiones de diseño. Revisión de secciones previas: objetivo general y objetivos específicos, requerimientos funcionales y no funcionales, resumen de las decisiones de stack al inicio de su sección, numeración y referencia de figuras y tablas con sus índices, y referencias bibliográficas.

1. Identificación del problema

1.1 Contexto

Librería Mi Sueño es un comercio de barrio en Mendoza que vende libros, útiles escolares, artística y regalería, y ofrece servicios de impresión, fotocopiado, anillado, plastificado y Rapipago. Toda la operación comercial hoy es presencial o telefónica: no existe presencia del negocio en internet más allá del contacto directo por WhatsApp.

El circuito de venta actual es enteramente manual: el cliente pregunta uno por uno qué productos hay disponibles, a qué precio y si hay stock, y el vendedor responde de memoria o revisando físicamente el local. No hay un catálogo consultable fuera del horario de atención ni una forma de que el cliente arme un pedido antes de escribir.

1.2 Problemas identificados

- **Sin catálogo público:** el negocio no tiene ninguna superficie donde un cliente pueda ver qué se vende y a qué precio, sin tener que preguntar.
- **Fricción en la consulta:** cada pedido implica una conversación de ida y vuelta por WhatsApp para relevar disponibilidad y precio, lo que consume tiempo del vendedor y desalienta pedidos grandes o de único ítem.
- **Nula visibilidad en buscadores:** al no existir un sitio, la librería no aparece en Google ni es descubrible por clientes nuevos del barrio o de Mendoza en general.
- **Dependencia total del trato presencial:** no hay ningún canal asincrónico donde el cliente arme y confirme un pedido fuera del horario de atención.
- **Sin registro único de la oferta:** no existe un listado único y actualizado de productos, precios y stock; la información vive en la cabeza del personal y en el local físico.
- **Crecimiento limitado por capacidad de atención humana:** cada consulta manual escala linealmente con el volumen de clientes, sin apalancamiento tecnológico.
- **Sin autonomía de gestión:** aun si existiera un catálogo online, el negocio necesita poder mantenerlo (altas, bajas, precios) sin depender de un desarrollador para cada cambio.

1.3 Objetivos del proyecto

A partir de los problemas identificados, el proyecto define un objetivo general, que expresa el propósito del trabajo, y un conjunto de objetivos específicos que lo desagregan en resultados concretos y verificables.

1.3.1 Objetivo general

Desarrollar un sistema web de comercio electrónico para Librería Mi Sueño, compuesto por una tienda pública con catálogo navegable y reserva de pedidos por WhatsApp, y por un panel de administración que permita al negocio gestionar su catálogo de forma autónoma, sobre una arquitectura desacoplada que admita incorporar en una etapa posterior pagos online y gestión de pedidos sin reconstruir lo ya desarrollado.

1.3.2 Objetivos específicos

Para alcanzar el objetivo general, el proyecto define los siguientes objetivos específicos:

1. Publicar un catálogo online navegable, con buscador, filtro por categorías y ficha de producto, accesible desde el celular en cualquier momento.
2. Permitir que el cliente arme un pedido (carrito) sin intervención del vendedor y lo derive a WhatsApp ya armado, para concretar la reserva.

3. Dar a la librería una gestión de contenido propia, mediante un panel de administración con login, para que el negocio cargue y mantenga el catálogo sin depender de un desarrollador para cada cambio.
4. Diseñar la arquitectura para que, en una etapa posterior, se puedan incorporar pagos online y gestión de pedidos sin reconstruir lo ya hecho.
5. Sostener una base técnica sólida y escalable, con separación clara entre las capas de presentación, lógica de negocio y datos.

2. Requerimientos identificados

A partir del análisis del funcionamiento actual del negocio y de los objetivos definidos en la sección 1, se identificaron los requerimientos que el sistema debe cumplir. Se presentan los más importantes, divididos en funcionales, que describen qué debe hacer el sistema, y no funcionales, que establecen las condiciones de calidad bajo las cuales debe hacerlo.

2.1 Requerimientos funcionales

La Tabla 2 presenta los requerimientos funcionales, agrupados según el área del sistema a la que pertenecen.

Tabla 2. *Requerimientos funcionales del sistema*

Código	Área	Requerimiento	Descripción
RF-01	Tienda pública	Página de inicio	Mostrar un buscador, los productos destacados y las novedades (hasta 5 de cada uno), el acceso a las categorías y un resumen de "Sobre nosotros".
RF-02	Tienda pública	Catálogo de productos	Listar los productos visibles con buscador que no distinga tildes, filtro por categoría y paginado.
RF-03	Tienda pública	Ficha de producto	Mostrar hasta 5 imágenes, título, categoría con breadcrumb, precio, stock, descripción y productos similares.
RF-04	Tienda pública	Carrito y reserva por WhatsApp	Agregar productos, modificar cantidades y ver el total; generar un mensaje con el detalle del pedido y enviarlo por WhatsApp.
RF-05	Tienda pública	Consulta por WhatsApp	Consultar por un producto puntual desde su ficha, con un mensaje de WhatsApp generado automáticamente.
RF-06	Tienda pública	Página institucional	Presentar la historia y la información del local en la página "Sobre nosotros".
RF-07	Acceso al panel	Autenticación	Iniciar y cerrar sesión con email y contraseña, recuperar el acceso con una clave temporal y cambiar la contraseña, de forma obligatoria tras recibir una clave temporal.
RF-08	Acceso al panel	Control de acceso por roles	Distinguir el rol administrador, con acceso total, del rol vendedor, limitado a la gestión del catálogo.
RF-09	Panel	Gestión de productos	Listar productos con buscador, filtros y paginado; darlos de alta, editarlos, darlos de baja y reactivarlos; marcarlos como oferta, novedad o destacado. El SKU lo genera el sistema.
RF-10	Panel	Gestión de imágenes	Cargar, reemplazar, reordenar y eliminar hasta 5 imágenes por producto, con procesamiento automático y reintento ante fallas.
RF-11	Panel	Gestión de categorías	Administrar categorías y subcategorías: alta, edición, baja lógica y reactivación.
RF-12	Panel (solo administrador)	Gestión de usuarios	Dar de alta usuarios con una clave temporal, editarlos, darlos de baja, reactivarlos y enviarles una nueva clave temporal por mail.

2.2 Requerimientos no funcionales

La Tabla 3 presenta los requerimientos no funcionales, junto con el atributo de calidad al que corresponde cada uno.

Tabla 3. *Requerimientos no funcionales del sistema*

Código	Atributo de calidad	Descripción
RNF-01	Usabilidad y adaptabilidad	La tienda y el panel deben ser 100 % adaptables y diseñados primero para celular, con puntos de corte en 360, 768, 1024 y 1440 px.
RNF-02	Rendimiento y posicionamiento	Las páginas públicas se generan en el servidor (SSR/ISR) para cargar rápido en conexiones móviles y ser indexadas por los buscadores; las imágenes se sirven optimizadas en formato WebP y en tres tamaños.
RNF-03	Seguridad	Contraseñas almacenadas con Argon2id, HTTPS obligatorio, access token de corta duración en memoria y refresh token en cookie HttpOnly, límite de solicitudes en los endpoints sensibles y validación de todos los datos de entrada.
RNF-04	Integridad de los datos	Las reglas críticas se refuerzan en la propia base de datos: bajas lógicas, SKU y slug inmutables, categoría e imagen obligatorias por producto y al menos un administrador activo.
RNF-05	Capacidad	El sistema debe soportar un catálogo de hasta 10.000 productos y alrededor de 30.000 visitas mensuales.
RNF-06	Disponibilidad	El backend debe permanecer siempre activo, sin suspensión por inactividad, para responder consultas y ejecutar las tareas programadas.
RNF-07	Mantenibilidad	Frontend y backend independientes, TypeScript en todo el stack y backend organizado con Clean Architecture, de modo que cada parte pueda modificarse sin afectar a las demás.
RNF-08	Extensibilidad	Debe poder incorporar pagos online y gestión de pedidos sin modificar las tablas existentes ni la arquitectura, apoyándose en una API versionada.

3. Propuesta de solución

3.1 Visión general

La solución es un e-commerce de catálogo con reserva por WhatsApp: el cliente navega productos, arma un carrito y el sistema genera un mensaje de WhatsApp con el detalle del pedido para que el vendedor lo confirme. El negocio administra el catálogo desde un panel propio, con login, sin depender de terceros para cada cambio.

El proyecto se plantea en dos horizontes: una primera implementación (MVP) que resuelve el problema completo de catálogo, reserva y autogestión de contenido, y una mejora futura que suma pagos online y gestión de pedidos sobre la misma base ya construida.

3.2 Alcance de la primera implementación (MVP)

- **Tienda pública:** home con buscador, productos destacados y novedades (hasta 5 de cada uno), acceso a las categorías y resumen de "Sobre nosotros"; los productos en oferta se identifican con su precio promocional.
- **Catálogo de productos:** buscador que no distingue tildes, filtro por categoría, paginado de 20 productos por página, y cada producto mostrado con foto, nombre y precio.
- **Ficha de producto:** galería de hasta 5 imágenes, título, categoría con breadcrumb, precio, stock, descripción, productos similares, y botones de agregar al carrito y consultar por WhatsApp. La tienda solo muestra productos activos, no dados de baja y con al menos una imagen procesada.
- **Carrito de compras:** accesible desde cualquier página, con cantidades, precios y total; se guarda en el navegador del cliente, sin registro ni tablas en la base, y aclara que precios y stock se confirman por WhatsApp; un botón arma el mensaje de reserva y lo envía por WhatsApp.
- **Página institucional:** "Sobre nosotros", con historia e información del local.
- **Encabezado y pie de página:** presentes en todas las pantallas, con navegación, botón de WhatsApp, redes sociales y dirección.
- **Panel de administración:** acceso privado con email y contraseña, con dos roles activos (administrador y vendedor); listados de productos y categorías con buscador, filtro por estado y paginado; alta y edición de productos (nombre, descripción, precio, stock, estado activo/inactivo, hasta 5 imágenes, categorías con una principal y productos similares), con SKU generado automáticamente por el sistema; marcado de producto como oferta, novedad o destacado; alta, edición y baja de categorías; baja lógica con posibilidad de reactivación de productos y categorías (no se contempla la baja definitiva).
- **Gestión de imágenes:** carga, reemplazo, reordenamiento y borrado de las imágenes de cada producto; el sistema conserva el original en un almacenamiento privado y genera en segundo plano versiones optimizadas en formato WebP (400, 800 y 1200 px), con reintento ante fallas de procesamiento.
- **Usuarios y acceso:** el administrador da de alta a los usuarios con una clave temporal que deben cambiar en su primer ingreso, los edita, los da de baja, los reactiva y puede enviarles una nueva clave temporal por mail; todo usuario puede cambiar su propia contraseña y recuperar el acceso con "olvidé mi contraseña" (clave temporal de un solo uso, con vencimiento de 30 minutos). El sistema nunca puede quedar sin un administrador activo.
- **Envío de mails:** correos transaccionales con las claves temporales, tanto de recuperación de contraseña como de reseteo realizado por el administrador.
- **Base de datos propia:** modelo relacional de siete tablas (usuarios, sesiones, categorías, productos, relación producto-categoría, productos similares e imágenes), con los roles administrador y vendedor activos desde el MVP y reglas de integridad reforzadas en la propia base: bajas lógicas, SKU y slug inmutables, al menos una categoría y una imagen por producto, máximo 5 imágenes y al menos un administrador activo.
- **Diseño 100 % adaptable:** tienda y panel usables en celular, tablet y computadora.

3.3 Mejora futura (fuera del alcance del MVP)

Quedan fuera de la primera implementación, como mejoras a incorporar más adelante sobre la misma base:

- **Pagos online con Mercado Pago:** tarjeta de crédito o débito con cuotas, dinero en cuenta o transferencia, gestionados íntegramente en el entorno de Mercado Pago.
- **Carrito con doble camino:** "Pagar ahora" (online) o "Reservar por WhatsApp", conviviendo en el mismo carrito.
- **Gestión de pedidos:** cada compra registrada con productos, cantidades, datos del comprador, medio de pago y estado (pendiente, pagado, entregado, cancelado).
- **Stock automático:** descuento de stock al confirmarse un pago, con bloqueo de compra cuando no hay disponibilidad.
- **Avisos por correo:** confirmación de compra al cliente y aviso de venta al negocio, reutilizando el servicio de envío de mails ya integrado en el MVP.
- **Estadísticas básicas:** ventas del período, productos más vendidos y visitas a la tienda.

3.4 Relación entre el MVP y la mejora futura

El MVP no es una versión reducida que luego se descarta: es la base funcional y de datos sobre la que se apoya la mejora futura. El modelo de datos del MVP no incluye tablas de pedidos ni de pagos, para no sumar al alcance inicial entidades sin uso; en la mejora futura se agregan como tablas nuevas, relacionadas con los productos y usuarios existentes, mediante nuevas migraciones y sin modificar las tablas ya creadas. A su vez, la separación entre frontend y backend definida en la sección 4 y el versionado de la API permiten agregar los endpoints de pago y las pantallas de gestión de pedidos sin modificar la arquitectura ni reescribir lo ya construido.

4. Definición del stack tecnológico

El sistema se construye con una arquitectura desacoplada: un frontend y un backend independientes, cada uno con su propio repositorio, ciclo de despliegue y responsabilidad. Esta separación es un requisito de diseño desde el inicio del proyecto, no una evolución posterior. La Tabla 4 resume la tecnología elegida para cada capa y su rol en el sistema.

Tabla 4. Stack tecnológico por capa

Capa	Tecnología	Rol
Frontend	Next.js (React) + TypeScript, alojado en Vercel	Tienda pública y panel de administración, servidos en el dominio principal. SSR/ISR para SEO y velocidad de carga.
Backend / API	NestJS + Drizzle ORM, alojado en Render (plan pago)	API REST propia versionada (/api/v1), expuesta en el subdominio api: catálogo, autenticación y usuarios del panel, procesamiento de imágenes, envío de mails y — en la mejora futura — pedidos y pagos.
Base de datos	PostgreSQL relacional en Supabase	Usuarios, sesiones, categorías, productos, imágenes y productos similares, en un schema propio no expuesto por la Data API de Supabase; en la mejora futura, pedidos y pagos en tablas nuevas.
Almacenamiento de archivos	Buckets de Supabase Storage	Original de cada imagen en un bucket privado y versiones optimizadas (WebP) en un bucket público, servidas por URL pública.
Mensajería con el cliente	WhatsApp vía deep links (wa.me)	Reserva, consulta y contacto, sin costo ni integración paga.
Envío de mails	Resend	Claves temporales de acceso al panel: recuperación de contraseña y reseteo por el administrador.
Dominio	.com.ar (NIC Argentina)	Sitio en el dominio principal y API en el subdominio api; requisito para la cookie de sesión y para el envío de mails.
Pagos (mejora futura)	SDK de Mercado Pago, integrado desde el backend Nest	Checkout Pro, confirmación de pago vía webhook contra la API propia.

4.1 Arquitectura de referencia

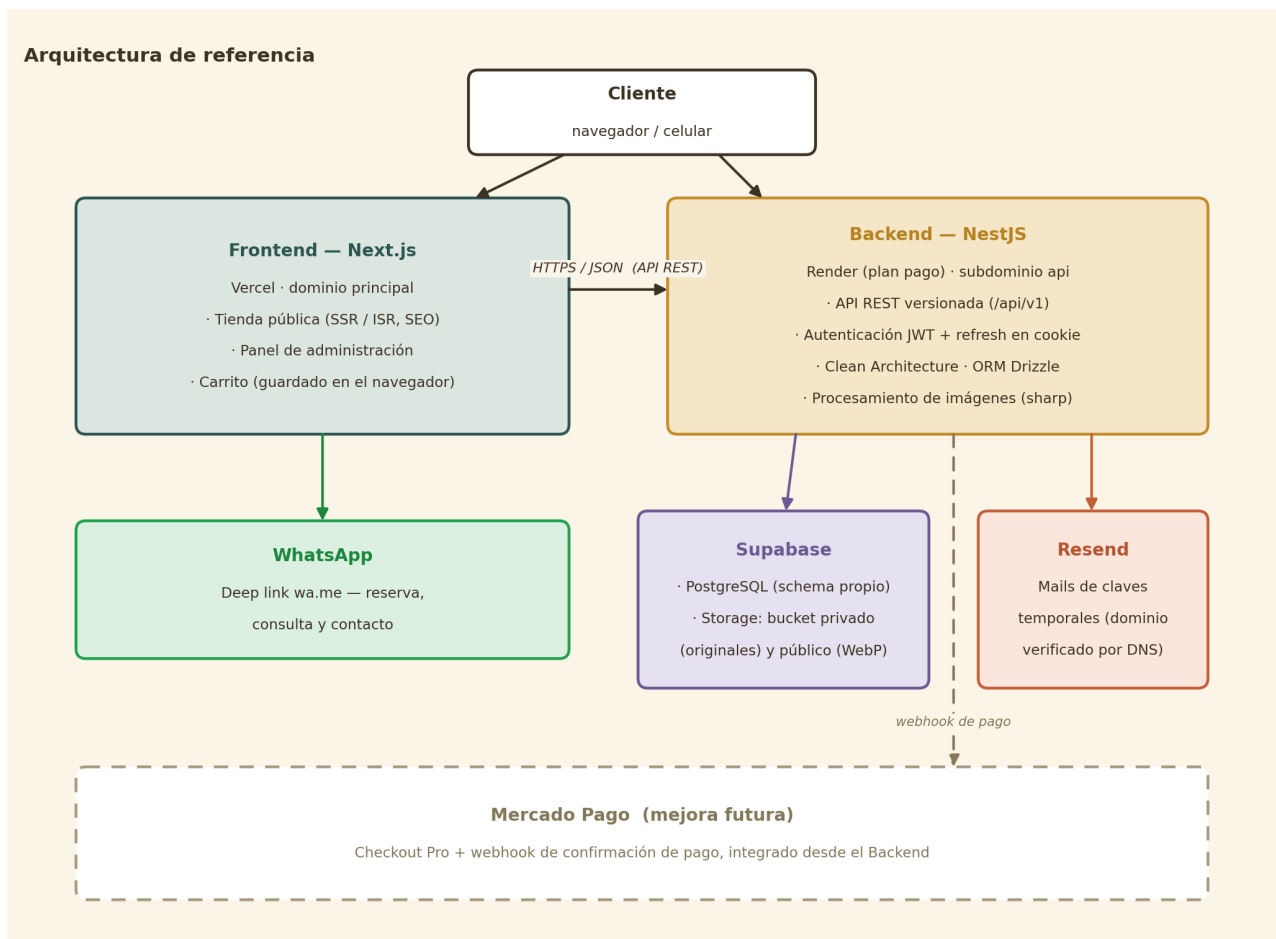


Figura 1. Arquitectura de referencia del sistema

La Figura 1 muestra la arquitectura de referencia del sistema. El frontend (Next.js en Vercel), publicado en el dominio principal, es responsable exclusivamente de la interfaz: tienda pública y panel de administración. El backend (NestJS en Render), publicado en el subdominio api del mismo dominio, concentra toda la lógica de negocio, la autenticación, el acceso a datos, el procesamiento de imágenes y el envío de mails (mediante Resend), y expone una API REST versionada bajo /api/v1 que el frontend consume por HTTPS. La base de datos y el almacenamiento de archivos viven en Supabase, fuera de ambos servicios de cómputo. Mercado Pago se integra en la mejora futura, exclusivamente desde el backend.

5. Toma de decisiones de stack fundamentadas

Cada pieza del stack definido en la sección 4 se eligió comparándola contra alternativas concretas. Se documentan acá los motivos, no solo la elección, para que la decisión sea auditable y revisable en entregas futuras.

5.1 Resumen de decisiones

La Tabla 5 resume las decisiones de stack del proyecto: la tecnología elegida para cada aspecto, su fundamento principal y la subsección en la que se desarrolla el análisis de alternativas correspondiente. Las subsecciones siguientes presentan ese detalle en el mismo orden.

Tabla 5. Resumen de las decisiones de stack

Decisión	Elegido	Fundamento principal	Detalle
Frontend	Next.js / Vercel	SEO, velocidad móvil, ecosistema para el panel de administración.	5.2
Backend	NestJS, servicio independiente	Escalado independiente, estructura modular, mantenibilidad a lo largo de varias entregas.	5.3
Hosting backend	Render (plan pago)	Procesos persistentes, tareas programadas, precio predecible, mismo flujo de CI/CD que Vercel.	5.4
ORM	Drizzle	Control fino de SQL, runtime liviano, ajuste al modelo relacional del proyecto.	5.5
Base de datos	PostgreSQL en Supabase	Relacional, transaccional, necesaria para stock y pedidos en la mejora futura.	5.6
Almacenamiento	Buckets de Supabase Storage	Mismo proveedor que la base de datos; bucket privado para originales y público para las versiones optimizadas.	5.6
Autenticación	JWT propio (Passport) + refresh token en cookie	Access token de 15 minutos en memoria y refresh token rotativo en cookie HttpOnly; sin depender de un proveedor de identidad externo.	5.7
Contraseñas	Argon2id	Algoritmo recomendado actualmente para almacenar contraseñas, resistente a ataques de fuerza bruta.	5.8
Envío de mails	Resend	SDK oficial para Node.js y plan gratuito suficiente para el bajo volumen de claves temporales.	5.9

5.2 Frontend: Next.js en Vercel

- SEO y velocidad de carga: crítico para que las fichas de producto aparezcan en Google y para que la tienda cargue rápido en conexiones de celular, escenario de uso principal del negocio.
- Un solo framework para tienda pública y panel de administración, lo que reduce la superficie de mantenimiento del lado del cliente.
- Ecosistema grande de React, con componentes (shadcn/ui, Radix) disponibles para acelerar la construcción del panel de administración.
- Vercel da CDN global, HTTPS, previews por rama y despliegue continuo desde GitHub sin configuración adicional, con plan gratuito suficiente durante el desarrollo.

Alternativas descartadas: una SPA sin metaframework (Vite) simplifica el build pero pierde el SSR/ISR que sostiene el SEO requerido. Remix es comparable en capacidades, pero tiene un ecosistema de componentes y de hosting gestionado más pequeño que el de Next.js/Vercel para este tamaño de equipo.

5.3 Backend: NestJS como servicio independiente

El backend se construye como una aplicación independiente del frontend, no como funciones o rutas dentro del mismo proyecto Next.js. Los motivos:

- **Separación de responsabilidades y de escalado:** el frontend (contenido, SEO, renderizado) y el backend (lógica de negocio, validaciones, integraciones de pago) tienen perfiles de carga y de despliegue distintos. Separarlos permite escalar o reiniciar uno sin afectar al otro, y evita que un pico de tráfico de la tienda pública compita por recursos con la API.
- **Estructura para crecer en alcance:** a medida que se suman autenticación con roles, gestión de usuarios, procesamiento de imágenes y envío de mails en el MVP, y webhooks de Mercado Pago y lógica de stock en la mejora futura, un backend estructurado con módulos, inyección de dependencias y testing de primera clase (NestJS) escala mejor en mantenibilidad que funciones sueltas sin framework.
- **Portabilidad:** un backend Nest corriendo como servicio Node estándar (no como función serverless atada a un proveedor) se puede mover a cualquier plataforma con soporte de contenedores o de Node sin reescribir código.
- **NestJS puntualmente:** impone una estructura modular (controllers, services, DTOs, guards) con soporte nativo de TypeScript y decoradores de validación (class-validator), que mapea limpio contra el dominio del proyecto (productos, categorías, usuarios, y en la mejora futura pedidos y pagos).
- **Arquitectura interna:** el backend se organiza con Clean Architecture en cuatro capas (dominio, aplicación, infraestructura y presentación), con dependencias que apuntan siempre hacia el dominio, y en dos módulos: identity (usuarios, autenticación y sesiones) y catalog (categorías, productos, imágenes y búsqueda). Las reglas de negocio quedan aisladas del framework, de la base de datos y de los servicios externos, por lo que un proveedor (por ejemplo, el de mails) puede reemplazarse sin modificar la lógica.

Alternativas consideradas y descartadas: Express o Fastify sin framework encima son más livianos pero delegan toda la estructura (módulos, inyección de dependencias, validación) al criterio de cada desarrollador, lo que en un proyecto de varias entregas tiende a acumular inconsistencia. Construir el backend como rutas de API dentro del propio proyecto Next.js se descartó por el punto de escalado y mantenibilidad recién explicado: esa alternativa mezclaría dos responsabilidades distintas en un solo despliegue.

5.4 Hosting del backend: Render

- Soporta servicios Node de larga duración (no solo funciones serverless de ejecución corta), lo que encaja con NestJS: un proceso persistente con su propio ciclo de vida, conexiones a base de datos con pool, tareas programadas y procesamiento de imágenes en segundo plano y, en la mejora futura, procesamiento de webhooks que necesita responder rápido sin cold start recurrente.
- Despliegue continuo desde GitHub, HTTPS gratuito y variables de entorno gestionadas — mismo flujo operativo que Vercel para el frontend, sin curva de aprendizaje adicional.
- Modelo de precios simple y predecible por servicio, más fácil de presupuestar que un modelo de facturación por uso.
- Se requiere el plan pago: el plan gratuito apaga el servicio tras un período de inactividad, lo que además de demorar la primera respuesta frena las tareas programadas (depuración de sesiones vencidas y reproceso de imágenes) y el procesamiento de imágenes en segundo plano. Por eso el backend se aloja en un plan pago, siempre activo.

Alternativas consideradas: Railway ofrece una experiencia similar con precios por uso, lo que hace más difícil dar una cifra mensual fija; Fly.io da más control de infraestructura (regiones, máquinas) a costa de más complejidad operativa, que no se justifica para el volumen de tráfico esperado en un comercio de este tamaño. Ejecutar el

backend como funciones serverless en el propio Vercel se descartó por el punto de separación de responsabilidades de 5.3.

5.5 ORM: Drizzle en lugar de Prisma

La Tabla 6 compara Drizzle y Prisma según los criterios relevantes para el proyecto.

Tabla 6. Comparación entre Drizzle y Prisma

Criterio	Drizzle	Prisma
Modelo mental	SQL tipado: las queries se escriben muy cerca del SQL real, con autocompletado y tipos inferidos.	DSL propio (schema.prisma) que genera un cliente; abstrae más el SQL subyacente.
Rendimiento / arranque	Sin motor de query engine binario aparte; queries compiladas a SQL directo, arranque más liviano.	Depende de un query engine empaquetado junto a la app; arranque algo más pesado.
Control fino	Alto: útil para las consultas de catálogo con filtros, categorías, paginado y búsqueda sin distinguir tildes.	Bueno para CRUD estándar; consultas muy específicas a veces requieren SQL crudo igual.
Migraciones	drizzle-kit genera SQL versionado, legible y editable a mano si hace falta; admite migraciones SQL manuales para funciones, triggers e índices especiales.	Prisma Migrate, maduro y muy usado, pero el SQL generado es menos editable en la práctica.
Curva de adopción	Requiere pensar en términos de SQL relacional.	Más rápido de empezar para quien no viene de SQL.

Se elige Drizzle porque el modelo de datos del proyecto (usuarios, sesiones, categorías, productos, imágenes y, en la mejora futura, pedidos y pagos) es relacional y se apoya en reglas implementadas en la propia base (triggers, restricciones diferidas e índices de búsqueda por similitud), que Drizzle permite incorporar mediante migraciones SQL manuales versionadas junto con el resto del schema. Además, se prioriza un runtime liviano en Render y queries explícitas por sobre la velocidad de adopción que da la abstracción de Prisma. La contrapartida que suele señalarse —una curva de entrada algo mayor para quien no viene de SQL— no afecta a este proyecto, ya que el equipo conoce de antemano la herramienta y trabaja con TypeScript estricto en todo el stack (ver 6.1).

5.6 Base de datos y almacenamiento: Supabase

- Un único proveedor resuelve PostgreSQL administrado y almacenamiento de archivos (bucket) con el mismo panel, las mismas credenciales de proyecto y una sola relación comercial, lo que simplifica la operación.
- Las imágenes se guardan en dos buckets: el original en uno privado y las versiones optimizadas en uno público. La base de datos guarda solo las claves de los archivos, no las URL, lo que permite cambiar de dominio o de CDN sin migrar datos.
- Las tablas se crean en un schema propio, no expuesto por la Data API de Supabase: todo acceso a los datos pasa por la API propia, que concentra la autenticación y las reglas de negocio.
- PostgreSQL es la base natural para un modelo relacional de catálogo (productos, categorías, imágenes, y en la mejora futura pedidos), con soporte completo de claves foráneas, restricciones únicas, triggers y transacciones —necesarias hoy para reglas como la categoría e imagen obligatorias por producto, y en la mejora futura para el descuento de stock atómico al confirmarse un pago.

- El plan gratuito (500 MB de base de datos y 1 GB de storage; Supabase, s.f.) alcanza para el desarrollo y la carga inicial del catálogo. Para producción se prevé el plan pago, ya que el gratuito pausa el proyecto tras una semana de inactividad y no incluye backups; el cambio de plan no requiere migrar de proveedor.

Alternativa considerada: una base de datos administrada en Render evitaría un proveedor adicional, pero obligaría a resolver el almacenamiento de imágenes en otro servicio aparte, porque Render no está pensado como CDN de archivos estáticos. Concentrar base de datos y bucket en Supabase es la opción que minimiza la cantidad de proveedores sin sacrificar capacidades.

5.7 Autenticación: JWT propio y refresh token en cookie

La autenticación del panel se implementa en el propio backend, con Passport y su estrategia JWT, en lugar de delegarse en un servicio externo. Los motivos:

- **Reglas de acceso propias:** el alta con clave temporal, el cambio obligatorio de contraseña, la recuperación con una clave de un solo uso, la protección del último administrador y la revocación de sesiones se implementan en el backend y en la base de datos, sin depender de lo que permite un proveedor.
- **Una sola fuente de verdad:** usuarios, roles y sesiones viven en las tablas users y sessions del schema propio, por lo que el rol y el estado se consultan en cada solicitud y cualquier cambio tiene efecto inmediato.
- **Esquema de dos tokens:** un access token JWT de 15 minutos que el frontend conserva solo en memoria y un refresh token rotativo, con detección de reutilización, en una cookie HttpOnly, Secure y SameSite=Strict, lo que reduce el impacto de un eventual robo de credenciales (ver 8.7).
- **Integración nativa con NestJS:** Passport y su estrategia JWT se integran con los guards del framework, que resuelven la autenticación y la autorización por roles en toda la API.

Alternativas consideradas: delegar la autenticación en un servicio externo, como el de Supabase, reduciría el código propio, pero separaría las cuentas del modelo de datos del panel y obligaría a adaptar a ese servicio reglas como la clave temporal o la protección del último administrador. Auth.js se descartó porque gestiona la sesión dentro de la aplicación Next.js, mientras que en esta arquitectura la autenticación debe resolverse en la API, que es la que protege los datos. Guardar ambos tokens en localStorage simplificaba la implementación y no requería dominio compartido, pero cualquier script ejecutado en la página podría leerlos.

5.8 Contraseñas: Argon2id

- **Recomendación vigente:** Argon2id es el algoritmo que OWASP recomienda como primera opción para almacenar contraseñas (OWASP Foundation, s.f.).
- **Resistencia a ataques por fuerza bruta:** además de tiempo de cómputo, cada cálculo exige una cantidad configurable de memoria, lo que encarece los ataques con hardware especializado.
- **Sin contraseñas en texto plano:** la contraseña se valida (entre 8 y 128 caracteres) y se transforma en hash antes de guardarse; el dominio del backend trabaja solo con hashes.
- **Criterio distinto para los tokens:** los refresh tokens se guardan con SHA-256, ya que al ser aleatorios y de alta entropía un hash lento no agrega seguridad y sí costo (ver 8.4).

Alternativa considerada: bcrypt es un algoritmo maduro y muy difundido, pero OWASP lo reserva para sistemas heredados en los que Argon2id no está disponible, y la mayoría de sus implementaciones limita la entrada a 72 bytes (OWASP Foundation, s.f.), lo que obligaría a restringir la longitud máxima de las contraseñas por debajo de los 128 caracteres admitidos.

5.9 Envío de mails: Resend

- **Volumen muy bajo:** en el MVP solo se envían claves temporales, tanto por recuperación de contraseña como por reseteo realizado por un administrador.
- **Integración simple:** Resend ofrece un SDK oficial para Node.js, que se integra directamente con NestJS.
- **Plan gratuito suficiente:** incluye 3.000 emails por mes, con un máximo de 100 por día y un dominio verificado, capacidad holgada para el volumen previsto (Resend, s.f.).
- **Proveedor reemplazable:** el envío se encapsula dentro del contrato PasswordResetNotifier, por lo que cambiar de proveedor solo requiere una nueva implementación, sin modificar la lógica de negocio (ver 8.4).

Requisito: para enviar mails a cualquier destinatario, Resend exige verificar el dominio propio mediante registros DNS (SPF y DKIM), por lo que depende del registro previo del dominio .com.ar (ver 6.2). Alternativas consideradas: otros servicios de correo transaccional ofrecen capacidades equivalentes; la elección se basó en la simplicidad de integración y en que el plan gratuito cubre el volumen del MVP. Como el proveedor queda aislado detrás de un contrato, la decisión puede revisarse sin impacto en el resto del sistema.

6. Análisis de viabilidad

6.1 Viabilidad técnica

- Las tres tecnologías elegidas (Next.js, NestJS, PostgreSQL/Drizzle) son maduras, con documentación extensa y comunidades grandes; no hay apuesta a tecnología experimental en ningún punto del stack.
- Sin curva de aprendizaje: el equipo conoce de antemano las tecnologías que conforman el stack (entre ellas Next.js, NestJS, Drizzle y PostgreSQL), por lo que su adopción no requiere tiempo de capacitación y el esfuerzo del proyecto se concentra en el diseño y la construcción del sistema.
- El volumen esperado para un comercio de este tamaño (un catálogo de hasta 10.000 productos y alrededor de 30.000 visitas mensuales) está muy por debajo de los límites de cualquiera de los tres proveedores elegidos en sus planes de entrada.
- El equipo debe manejar dos repositorios y dos despliegues (frontend y backend) en lugar de uno, lo que agrega coordinación pero es un patrón estándar de la industria, con tooling maduro (CI/CD de GitHub, variables de entorno por servicio) para sostenerlo.
- Riesgo técnico principal: CORS y autenticación entre dos servicios distintos (Vercel y Render). Se mitiga con CORS explícito en Nest, limitado al dominio del frontend y con envío de credenciales habilitado, y con un esquema de dos tokens: un access token JWT de 15 minutos que el frontend conserva solo en memoria y un refresh token rotativo, con detección de reuso, en una cookie HttpOnly, Secure y SameSite=Strict. Esta cookie exige que frontend y API compartan el dominio registrado (el sitio en el dominio principal y la API en el subdominio api), por lo que el registro del dominio .com.ar es un requisito para la puesta en producción.

6.2 Riesgos y mitigaciones

La Tabla 7 identifica los principales riesgos del proyecto, su impacto y la mitigación prevista para cada uno.

Tabla 7. Riesgos del proyecto y sus mitigaciones

Riesgo	Impacto	Mitigación
Dos proveedores de hosting (Vercel + Render) en vez de uno	Bajo — más superficie operativa, mismo modelo de CI/CD en ambos	Documentar variables de entorno y URLs de cada servicio en un README de infraestructura del proyecto
El plan gratuito de Render hace spin-down del backend tras inactividad	Alto — además de demorar la primera consulta, frena las tareas programadas y el procesamiento de imágenes	Contratar el plan pago de Render (servicio siempre activo) antes de publicar en el dominio propio
CORS y autenticación entre dominios distintos	Bajo — mitigable con configuración estándar	CORS explícito en Nest con credenciales habilitadas, access token JWT en memoria y refresh token en cookie HttpOnly con SameSite=Strict y verificación de origen
El panel de administración maneja credenciales y datos del negocio desde el primer día	Medio — superficie de seguridad mayor que un sitio sin login	Contraseñas con hash (Argon2id), límite de solicitudes en el login y en la recuperación de contraseña, revocación de sesiones ante cambios de clave, rol o estado, HTTPS obligatorio y validación estricta (Zod / class-validator) en cada endpoint
Volumen de imágenes y consultas crece más rápido de lo esperado	Bajo — los planes gratuitos tienen límites de almacenamiento y de cómputo	Camino de upgrade a planes pagos en Supabase y Render sin migración de proveedor

Riesgo	Impacto	Mitigación
Dependencia del dominio propio (.com.ar)	Medio — sin el dominio registrado no funcionan la cookie de sesión ni el envío de mails	Registrar el dominio en NIC Argentina y verificarlo en Resend (registros DNS SPF y DKIM) antes de la puesta en producción
Límites del plan gratuito de Resend	Bajo — solo se envían mails de claves temporales, con volumen muy bajo	Límite de solicitudes en la recuperación de contraseña; proveedor reemplazable sin modificar la lógica de negocio

6.3 Conclusión

El stack definido (Next.js/Vercel para el frontend, NestJS/Drizzle/Render para el backend, PostgreSQL y buckets en Supabase, Resend para el envío de mails) es técnicamente sólido, usa herramientas maduras y mantiene el volumen esperado del proyecto muy por debajo de los límites de cada proveedor en sus planes de entrada. La viabilidad queda sujeta a dos condiciones previas a la puesta en producción: registrar el dominio [.com.ar](https://nic.ar/), necesario para la cookie de sesión y para la verificación del envío de mails, y contratar el plan pago de Render, que mantiene el backend siempre activo. La separación entre frontend y backend, definida desde el inicio del proyecto, deja la base preparada para incorporar la mejora futura (pagos, pedidos, stock automático y estadísticas) sin reescribir lo ya construido.

7. Bases de Datos

Esta sección describe el modelo de datos del MVP: las tablas que lo componen, cómo se relacionan entre sí, las reglas de negocio que se garantizan en la propia base y los fundamentos de cada decisión de diseño. El modelo se implementa en PostgreSQL, alojado en Supabase, y su script de creación fue validado sobre PostgreSQL 16.

El diseño parte de un principio: la integridad de los datos no debe depender exclusivamente de la aplicación. Por eso, además de las validaciones del backend, las reglas más importantes se refuerzan en la base mediante restricciones, índices y triggers, de modo que se cumplan aun ante un error de programación o una modificación realizada directamente sobre la base.

7.1 Visión general del modelo

El modelo está compuesto por siete tablas, agrupadas en dos áreas funcionales: el acceso al panel de administración (usuarios y sesiones) y el catálogo (categorías, productos, imágenes y sus relaciones). Todas se crean en el schema libreria, separado del schema público de Supabase y no expuesto por su Data API, de modo que el único acceso a los datos es a través de la propia API. La Tabla 8 resume el propósito de cada una.

Tabla 8. Tablas del modelo de datos

Tabla	Área	Propósito
users	Acceso al panel	Cuentas del panel con rol ADMIN o SELLER, contraseña (como hash), estado y datos de auditoría.
sessions	Acceso al panel	Sesiones iniciadas a partir de un refresh token; permiten revocar el acceso de forma individual.
categories	Catálogo	Categorías y subcategorías que organizan los productos en la tienda y en los filtros.
products	Catálogo	Entidad central: datos comerciales, precio, stock y marcas de oferta, novedad y destacado.
product_categories	Catálogo	Relación muchos a muchos entre productos y categorías, con la indicación de la categoría principal.
related_products	Catálogo	Productos similares cargados a mano para cada producto, con su orden de aparición.
product_images	Catálogo	Hasta 5 imágenes por producto, con sus archivos y su estado de procesamiento.

Quedan deliberadamente fuera del modelo el carrito, que se guarda en el navegador del cliente y se confirma por WhatsApp, y los pedidos y pagos, que corresponden a la mejora futura (ver 7.9).

7.2 Diagrama entidad-relación

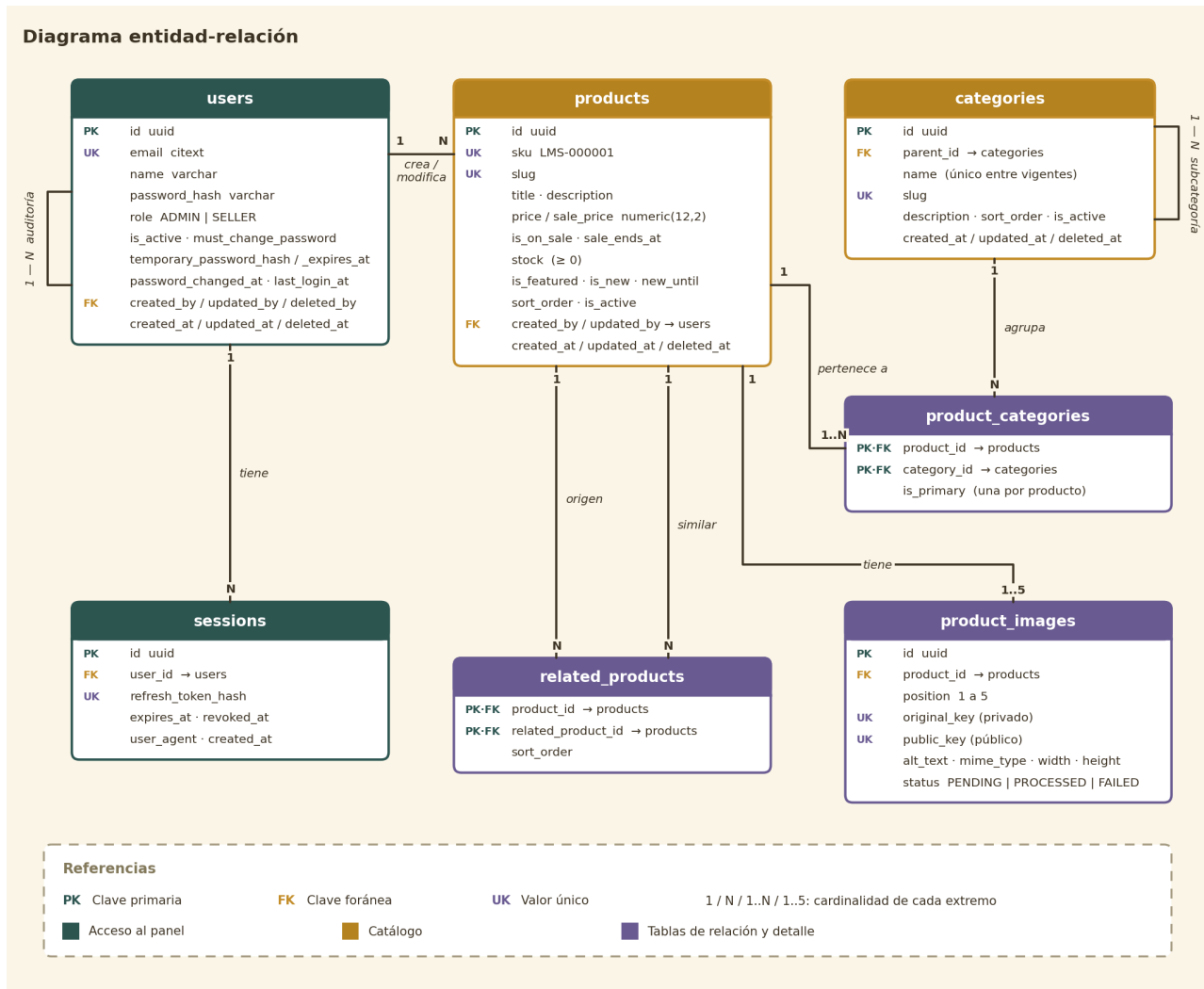


Figura 2. Diagrama entidad-relación del modelo de datos

La Figura 2 muestra cada tabla con sus columnas principales y las relaciones entre ellas. Los valores sobre cada línea indican la cardinalidad de cada extremo: por ejemplo, 1..5 en la relación entre products y product_images significa que cada producto tiene como mínimo una y como máximo cinco imágenes. Los colores agrupan las tablas según su función: acceso al panel, catálogo, y tablas de relación y detalle. El diagrama completo, junto con el script de creación de la base de datos (DDL), se encuentra en el repositorio del proyecto (ver Anexo 1.2).

7.3 Relaciones entre tablas

La Tabla 9 resume cada relación del modelo, su cardinalidad y el motivo por el que se modeló de esa forma.

Tabla 9. Relaciones entre las tablas del modelo

Relación	Cardinalidad	Fundamento
users → users	1 a N (reflexiva)	El alta, la modificación y la baja de un usuario las realiza otro usuario ADMIN; created_by, updated_by y deleted_by dejan registro de quién realizó cada acción.
users → sessions	1 a N	Un usuario puede tener varias sesiones abiertas (por ejemplo, en distintos dispositivos) y cada sesión pertenece a un único usuario.

Relación	Cardinalidad	Fundamento
users → products	1 a N	created_by y updated_by registran qué usuario cargó y modificó cada producto, sin duplicar sus datos.
categories → categories	1 a N (reflexiva)	parent_id permite definir subcategorías sin crear una tabla por nivel; una categoría sin padre es una categoría raíz.
products ↔ categories	N a N, mediante product_categories	Un producto puede pertenecer a varias categorías y una categoría agrupa muchos productos; is_primary define la categoría principal, usada en el breadcrumb y para completar los productos similares.
products ↔ products	N a N, mediante related_products	Relación reflexiva: cada fila vincula un producto con otro que se muestra como similar en su ficha. Necesita tabla propia porque se carga a mano y no se infiere solo por categoría.
products → product_images	1 a 1..5	Cada imagen pertenece a un solo producto; el mínimo de una y el máximo de cinco son reglas de negocio reforzadas en la base.

Acciones ante una eliminación física. Aunque el sistema solo aplica bajas lógicas, cada clave foránea define qué ocurre si una fila se elimina físicamente. Esto documenta la intención del modelo y lo protege ante operaciones manuales:

- **CASCADE:** las sesiones, las asignaciones de categorías, los productos similares y las imágenes se eliminan junto con su usuario o producto, porque no tienen sentido de forma independiente.
- **RESTRICT:** no se puede eliminar una categoría que tenga subcategorías o productos asociados, para no dejar productos sin clasificar ni jerarquías incompletas.
- **SET NULL:** las columnas de auditoría (created_by, updated_by y deleted_by) quedan vacías si se elimina el usuario referido, y el registro afectado se conserva.

7.4 Diccionario de datos

A continuación se detallan las columnas de cada tabla. Las columnas que cumplen una misma función se agrupan en una sola fila para facilitar la lectura. Las Tablas 10 a 16 presentan el diccionario de cada una de las siete tablas del modelo.

Tabla 10. Diccionario de datos de la tabla users: cuentas del panel de administración

Columna	Tipo	Descripción
id	uuid	Identificador único, generado por la base.
email	citext	Email de acceso. Se compara sin distinguir mayúsculas y es único entre los usuarios no dados de baja.
name	varchar(120)	Nombre para mostrar; no puede estar vacío.
password_hash	varchar(255)	Hash de la contraseña (Argon2id); la contraseña nunca se guarda en texto plano.
role	user_role	ADMIN o SELLER; por defecto, SELLER.
is_active	boolean	Si es falso, el usuario no puede iniciar sesión.

Columna	Tipo	Descripción
must_change_password	boolean	Obliga a cambiar la contraseña en el próximo ingreso (alta o reseteo realizado por un administrador).
temporary_password_hash / temporary_password_expires_at	varchar(255) / timestamptz	Clave temporal de "olvidé mi contraseña" y su vencimiento. Convive con la contraseña actual y ambas columnas se completan juntas.
password_changed_at / last_login_at	timestamptz	Fecha del último cambio de contraseña (registrada por trigger) y del último ingreso.
created_by / updated_by / deleted_by	uuid (FK users)	Administrador que creó, modificó o dio de baja al usuario.
created_at / updated_at / deleted_at	timestamptz	Fecha de alta, de última modificación (actualizada automáticamente por trigger) y de baja lógica.

Tabla 11. Diccionario de datos de la tabla *sessions*: sesiones iniciadas a partir de un refresh token

Columna	Tipo	Descripción
id	uuid	Identificador de la sesión.
user_id	uuid (FK users)	Usuario al que pertenece la sesión.
refresh_token_hash	varchar(255)	Hash del refresh token, único; el token real nunca se almacena.
expires_at	timestamptz	Vencimiento de la sesión; debe ser posterior a su creación.
revoked_at	timestamptz	Fecha de revocación: cierre de sesión, rotación del token, reuso detectado o cambios sensibles sobre el usuario.
user_agent	varchar(500)	Navegador o dispositivo desde el que se inició la sesión.
created_at	timestamptz	Fecha de inicio de la sesión.

Tabla 12. Diccionario de datos de la tabla *categories*: categorías y subcategorías del catálogo

Columna	Tipo	Descripción
id	uuid	Identificador único.
parent_id	uuid (FK categories)	Categoría padre; vacío si es una categoría raíz. Una categoría no puede ser su propio padre.
name	varchar(100)	Nombre visible; único, sin distinguir mayúsculas, entre las categorías no dadas de baja.
slug	varchar(120)	Identificador para la URL (minúsculas, números y guiones); único e inmutable.
description	text	Descripción opcional.

Columna	Tipo	Descripción
sort_order / is_active	integer / boolean	Posición en los listados y estado activo o inactivo, que permite ocultar la categoría sin darla de baja.
created_at / updated_at / deleted_at	timestampz	Fecha de alta, de última modificación (actualizada automáticamente por trigger) y de baja lógica.

Tabla 13. Diccionario de datos de la tabla *products*: entidad central del catálogo

Columna	Tipo	Descripción
id	uuid	Identificador único.
sku	varchar(64)	Código interno generado por el sistema con formato LMS-000001; único e inmutable.
title	varchar(200)	Nombre del producto; no puede estar vacío.
slug	varchar(220)	Identificador para la URL de la ficha; único e inmutable.
description	text	Descripción del producto; el modelo no incluye un subtítulo aparte.
price	numeric(12,2)	Precio regular; debe ser mayor a cero.
sale_price / is_on_sale / sale_ends_at	numeric(12,2) / boolean / timestampz	Precio de oferta (mayor a cero y menor al regular), marca de oferta, que exige precio de oferta, y vencimiento opcional.
stock	integer	Unidades disponibles; nunca puede ser negativo.
is_featured	boolean	Marca de producto destacado.
is_new / new_until	boolean / timestampz	Marca de novedad y su vencimiento opcional.
sort_order	integer	Posición en destacados, novedades y listados; a igual posición se ordena por fecha de creación descendente.
is_active	boolean	Estado Activo o Inactivo definido en el formulario del panel.
created_by / updated_by	uuid (FK users)	Usuario que cargó el producto y último usuario que lo modificó.
created_at / updated_at / deleted_at	timestampz	Fecha de alta, de última modificación (actualizada automáticamente por trigger) y de baja lógica.

Tabla 14. Diccionario de datos de la tabla *product_categories*: asignación de categorías a cada producto

Columna	Tipo	Descripción
product_id	uuid (PK, FK products)	Producto asignado.

Columna	Tipo	Descripción
category_id	uuid (PK, FK categories)	Categoría asignada.
is_primary	boolean	Indica la categoría principal; como máximo una por producto.

Tabla 15. Diccionario de datos de la tabla *related_products*: productos similares de cada producto

Columna	Tipo	Descripción
product_id	uuid (PK, FK products)	Producto en cuya ficha se muestra el similar.
related_product_id	uuid (PK, FK products)	Producto que se muestra como similar; debe ser distinto del producto de origen.
sort_order	smallint	Orden de aparición en la ficha.

Tabla 16. Diccionario de datos de la tabla *product_images*: imágenes de cada producto y su procesamiento

Columna	Tipo	Descripción
id	uuid	Identificador único.
product_id	uuid (FK products)	Producto al que pertenece la imagen.
position	smallint	Posición de 1 a 5, única dentro de cada producto.
original_key	varchar(500)	Clave del archivo original en el bucket privado; única.
public_key	varchar(500)	Clave de la versión optimizada en el bucket público; única y obligatoria una vez procesada la imagen.
alt_text	varchar(255)	Texto alternativo, para accesibilidad y posicionamiento en buscadores.
mime_type	varchar(50)	Formato del archivo original: image/jpeg, image/png o image/webp.
width / height / size_bytes	integer	Dimensiones y peso del archivo; si se informan, deben ser positivos.
status	image_status	Estado del procesamiento: PENDING, PROCESSED o FAILED.
created_at	timestampz	Fecha de carga.

7.5 Reglas de negocio garantizadas por la base

Las reglas siguientes se validan en el backend y, además, en la base de datos, que actúa como última barrera ante errores de la aplicación o modificaciones directas. La Tabla 17 detalla el mecanismo que implementa cada regla y su motivo.

Tabla 17. Reglas de negocio garantizadas por la base de datos

Regla	Mecanismo en la base	Motivo
Bajas lógicas en usuarios, categorías y productos	Columna deleted_at e índices únicos parciales que solo consideran registros vigentes	Conservar el historial y permitir la reactivación sin perder datos ni relaciones.
Email y nombre de categoría únicos entre registros vigentes	Índices únicos parciales uq_users_email_not_deleted y uq_categories_name_not_deleted	Permitir reutilizar un email o un nombre que pertenecía a un registro dado de baja.
SKU generado por el sistema	Secuencia product_sku_seq, función next_product_sku() y CHECK de formato	Obtener códigos correlativos y únicos, aun ante altas simultáneas, sin intervención del usuario.
Slug y SKU inmutables	Trigger con la función genérica prevent_immutable_update	Son identificadores externos (URL y referencia en el mensaje de WhatsApp) y no deben cambiar.
Precio positivo, oferta menor al precio regular y stock no negativo	Restricciones CHECK en products	Evitar datos comerciales inconsistentes.
Al menos una categoría por producto	Constraint trigger diferido check_product_has_category	Todo producto debe poder ubicarse en el catálogo y en el breadcrumb.
Una sola categoría principal por producto	Índice único parcial uq_product_categories_primary	Definir sin ambigüedad el breadcrumb y la base de los productos similares.
Entre 1 y 5 imágenes por producto	Constraint trigger diferido check_product_has_image, CHECK de posición 1 a 5 y UNIQUE (product_id, position)	Toda ficha tiene al menos una imagen y respeta el máximo acordado.
Toda imagen procesada tiene su versión pública	CHECK chk_product_images_processed_has_public_key	Evitar que la tienda intente mostrar una imagen sin archivo optimizado.
Siempre existe un administrador activo	Trigger prevent_last_admin_removal, con bloqueo ante operaciones simultáneas	Impedir que el panel quede sin nadie capaz de administrar usuarios.
Revocación de sesiones ante cambios sensibles	Trigger revoke_user_sessions al cambiar contraseña, rol, estado o baja	Que un cambio de permisos o de clave tenga efecto inmediato en todas las sesiones abiertas.
Un producto no es similar de sí mismo; una categoría no es su propio padre	CHECK chk_related_products_not_self y chk_categories_not_own_parent	Evitar relaciones sin sentido. La detección de ciclos más largos en la jerarquía la realiza el backend.

Por qué se usan validaciones diferidas. Las reglas de "al menos una categoría" y "al menos una imagen" no pueden verificarse en el momento de insertar el producto, porque sus categorías e imágenes se insertan inmediatamente después, dentro de la misma transacción. Por eso se implementan como constraint triggers diferidos: la base espera al final de la transacción (COMMIT) y recién entonces comprueba que el producto quedó completo. Con el mismo criterio, la restricción de posición única de las imágenes es diferible, lo que permite reordenarlas aunque dos imágenes compartan momentáneamente la misma posición.

7.6 Interacción entre tablas en las operaciones principales

Los siguientes casos muestran cómo intervienen las tablas en las operaciones más frecuentes del sistema:

- **Alta de producto:** en una única transacción se inserta la fila en products (con el SKU obtenido de la secuencia), sus filas en product_categories con una categoría principal, sus productos similares en related_products y sus imágenes en product_images con estado PENDING. Al confirmar la transacción, los triggers diferidos verifican que el producto tenga al menos una categoría y una imagen; si no se cumple, se revierte toda la operación.
- **Publicación en la tienda:** un producto se muestra solo si está activo, no fue dado de baja y tiene al menos una imagen en estado PROCESSED. Mientras sus imágenes se procesan, el producto existe en el panel pero todavía no aparece en el sitio.
- **Consulta del catálogo:** el listado filtra products por visibilidad mediante índices parciales, se vincula con product_categories para filtrar por categoría y con product_images para obtener la imagen principal. La búsqueda por título usa un índice trigram que no distingue tildes y tolera errores de tipeo.
- **Ficha de producto:** se obtiene el producto por su slug, su categoría principal para el breadcrumb, sus imágenes ordenadas por posición y sus productos similares; si los cargados a mano no alcanzan, se completan con productos de la categoría principal.
- **Inicio y renovación de sesión:** el inicio de sesión crea una fila en sessions con el hash del refresh token. Cada renovación revoca la sesión utilizada y crea una nueva (rotación); si se presenta un token ya revocado, se interpreta como un posible robo y se revocan todas las sesiones del usuario.
- **Cambios sobre un usuario:** al darlo de baja, desactivarlo o cambiar su rol o su contraseña, un trigger revoca todas sus sesiones abiertas. Si la operación deja al sistema sin administradores activos, otro trigger la rechaza.
- **Baja y reactivación:** la baja completa deleted_at sin eliminar las filas relacionadas; al reactivar, el registro recupera sus categorías, imágenes y relaciones intactas.

7.7 Decisiones de diseño fundamentadas

Cada decisión relevante del modelo se tomó comparándola con una alternativa concreta. La Tabla 18 resume la opción elegida, la alternativa descartada y el fundamento.

Tabla 18. Decisiones de diseño del modelo de datos

Decisión	Alternativa descartada	Fundamento
Identificadores UUID	Enteros autoincrementales	No revelan la cantidad de registros ni permiten recorrer recursos por identificadores consecutivos, y pueden generarse sin coordinación con la base.
Precios en numeric(12,2)	Tipos de punto flotante	Representación exacta de montos: evita errores de redondeo en precios y, a futuro, en los totales de los pedidos.
Baja lógica con deleted_at	Borrado físico	Permite reactivar registros, conserva la trazabilidad y evita romper referencias, como las de futuros pedidos hacia un producto dado de baja.
SKU generado por secuencia; slug y SKU inmutables	SKU ingresado a mano o editable	Evita duplicados y errores de carga, y mantiene válidos los enlaces compartidos y las referencias enviadas por WhatsApp.
Email en citext	varchar con normalización en cada consulta	La unicidad y las búsquedas no distinguen mayúsculas sin depender de que cada consulta normalice el texto.
Tipos enumerados (user_role, image_status)	Texto libre o tablas de referencia	Restringen los valores válidos en la propia base con un costo mínimo, ya que los valores son pocos y estables.

Decisión	Alternativa descartada	Fundamento
Búsqueda con pg_trgm y una versión inmutable de unaccent	Búsqueda de texto completo (tsvector)	Los títulos son cortos: la similitud por trigramas tolera tildes y errores de tipeo sin la complejidad de configurar diccionarios de idioma.
Claves de archivo en lugar de URL	Guardar la URL completa de cada imagen	Cambiar de dominio, de CDN o de bucket no requiere migrar datos.
Hash del refresh token	Guardar el token en texto plano	Un acceso indebido a la base no permite reutilizar sesiones.
Reglas en la base además de en la aplicación	Validar solo en el backend	Defensa en profundidad: la integridad se mantiene aun ante errores de la aplicación o cambios directos sobre la base.
Schema propio, no expuesto por la Data API	Usar el schema público de Supabase	Las tablas no quedan publicadas automáticamente; todo acceso pasa por la API propia y sus reglas de autorización.
Fechas con zona horaria (timestampz)	timestamp sin zona horaria	Vencimientos y auditoría consistentes, independientemente de la zona horaria del servidor.
Carrito fuera de la base	Persistir el carrito en tablas propias	El carrito vive en el navegador y se confirma por WhatsApp: en el MVP no requiere estado en el servidor.

7.8 Implementación, índices y migraciones

El schema de cada tabla se declara con Drizzle ORM, y drizzle-kit genera a partir de él las migraciones SQL versionadas. Los elementos que el ORM no representa de forma nativa —funciones, triggers, constraint triggers diferidos, la secuencia del SKU y los índices especiales— se incorporan como migraciones SQL manuales, versionadas junto con el resto. Las extensiones utilizadas (citext, pg_trgm y unaccent) se instalan en un schema separado, extensions.

- **Índices para la tienda:** índices parciales para los productos visibles, los destacados, las novedades y las ofertas, ordenados por sort_order y fecha de creación. Al incluir solo las filas vigentes, se mantienen pequeños aunque crezca la cantidad de bajas.
- **Índice de búsqueda:** índice GIN con trigramas sobre el título sin tildes, que permite buscar por coincidencia parcial sin recorrer toda la tabla.
- **Índices de relaciones:** índices sobre las claves foráneas más consultadas (categoría de cada producto, productos similares, autoría y sesiones activas por usuario).

7.9 Evolución prevista: pedidos y pagos

El modelo del MVP no incluye tablas de pedidos ni de pagos. Cuando se incorpore la mejora futura, se agregarán como tablas nuevas que referenciarán a las existentes, sin modificarlas. Varias decisiones actuales ya preparan esa evolución: los precios en numeric(12,2); el stock con restricción de no negatividad, base para un descuento atómico al confirmarse un pago; la baja lógica de productos, que evita que un pedido quede apuntando a un registro inexistente; y los identificadores UUID, que pueden exponerse en comprobantes o enlaces sin revelar volúmenes de venta.

8. Backend

Esta sección describe el diseño del backend: su arquitectura, la organización en módulos y capas, las responsabilidades de sus clases principales, los flujos más relevantes, la API REST que expone y las decisiones de diseño que lo fundamentan. El backend se desarrolla con NestJS y TypeScript, utiliza Drizzle ORM para el acceso a datos y opera sobre la base de datos descrita en la sección 7.

Diagramas de clases completos. Por su tamaño, los diagramas de clases detallados de cada módulo y capa no se reproducen en este documento, ya que no podrían leerse correctamente en una página. Las figuras de esta sección presentan una vista resumida, y los diagramas completos, con todos sus atributos, métodos y relaciones, están disponibles en el repositorio del proyecto (ver Anexo 1.2), donde GitHub los muestra directamente como diagramas.

8.1 Arquitectura en capas

Diagrama de referencia: sección «1. Capas y regla de dependencias».

El backend sigue los principios de Clean Architecture: se organiza en cuatro capas con una regla de dependencias única, según la cual las capas externas dependen de las internas y nunca al revés. De esta forma, las reglas de negocio quedan aisladas del framework, de la base de datos y de los servicios externos. La Figura 3 muestra las capas y sus dependencias, y la Tabla 19 detalla la responsabilidad de cada una.

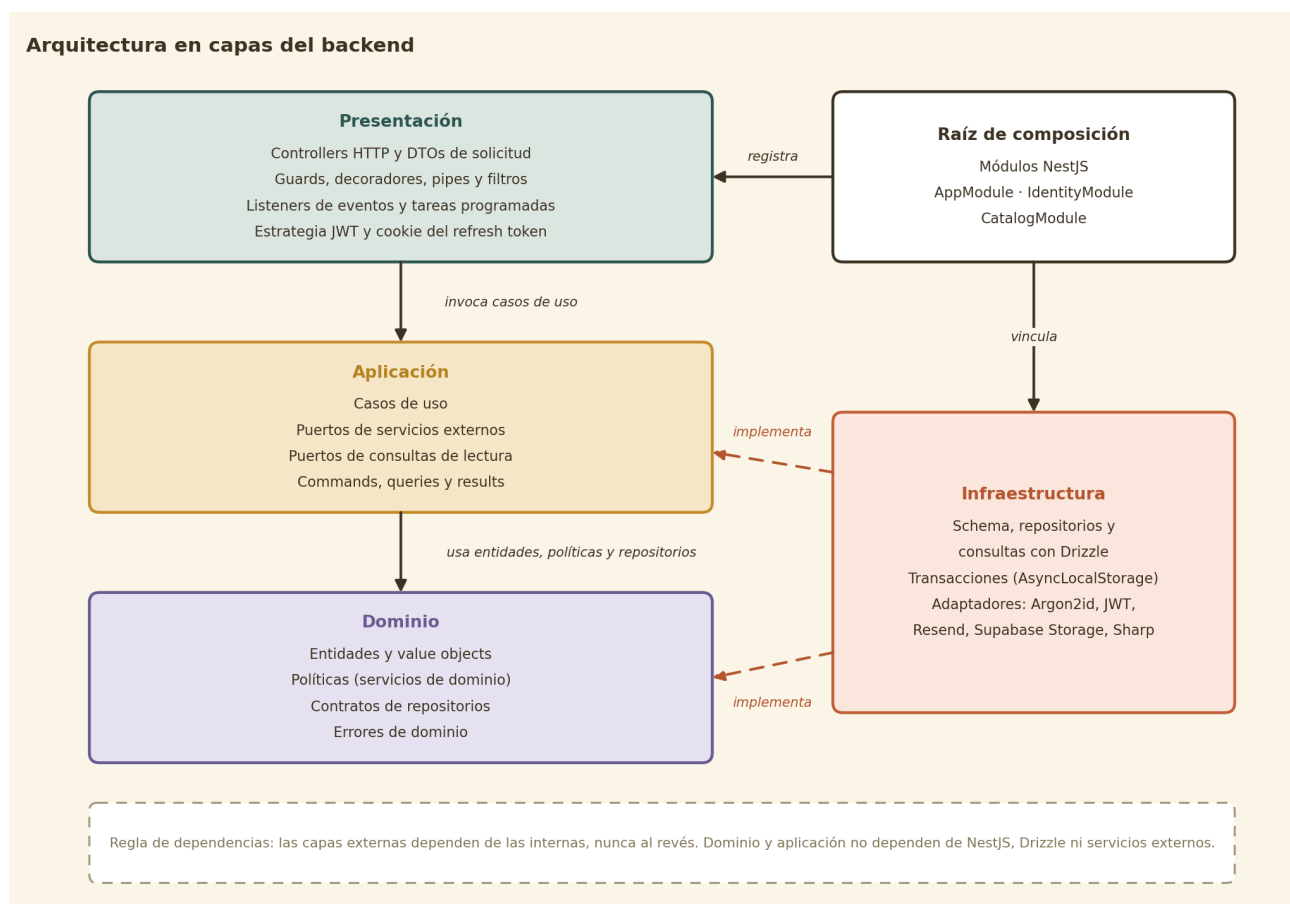


Figura 3. Arquitectura en capas del backend

Tabla 19. Capas del backend y sus responsabilidades

Capa	Responsabilidad	Contenido principal
Presentación	Punto de entrada: recibe las solicitudes HTTP, valida los datos y controla el acceso. No contiene lógica de negocio.	Controllers, DTOs, guards, decoradores, pipes, filtros, listeners de eventos y tareas programadas.
Aplicación	Orquesta cada operación del sistema (caso de uso) y define los contratos que necesita de servicios externos.	Casos de uso, puertos de servicios y de consultas, commands, queries y results.
Dominio	Núcleo del sistema: reglas de negocio independientes de cualquier tecnología.	Entidades, value objects, políticas, contratos de repositorios y errores de dominio.
Infraestructura	Implementa los contratos de las capas internas con tecnologías concretas.	Schema, repositorios y consultas con Drizzle; adaptadores de Argon2id, JWT, Resend, Supabase Storage y Sharp.
Raíz de composición	Vincula cada contrato con su implementación al iniciar la aplicación.	Módulos NestJS: AppModule, IdentityModule y CatalogModule.

Reglas de arquitectura. Para sostener esta organización a lo largo del proyecto se establecen las siguientes reglas:

- **Dirección única de dependencias:** el dominio no depende de ninguna otra capa; la aplicación depende solo del dominio; infraestructura y presentación dependen de las capas internas, sin depender entre sí.
- **Núcleo independiente del framework:** dominio y aplicación no utilizan NestJS, Drizzle ni otras librerías de infraestructura; los casos de uso son clases TypeScript sin dependencias externas, lo que facilita probarlos de forma aislada.
- **Contratos como clases abstractas:** repositorios y servicios externos se declaran como clases abstractas, que NestJS utiliza directamente para la inyección de dependencias y TypeScript verifica en tiempo de compilación.
- **Composición centralizada:** cada módulo NestJS es el único lugar donde se conocen a la vez los contratos y sus implementaciones.
- **Escritura y lectura separadas:** las operaciones de escritura trabajan con entidades y validan sus reglas; los listados y búsquedas usan puertos de consulta que devuelven los datos con el formato de cada pantalla, sin reconstruir entidades.
- **Control automático:** una herramienta de análisis estático (eslint-plugin-boundaries o dependency-cruiser) permite que la integración continua rechace importaciones que violen la dirección de las dependencias.

8.2 Módulos del sistema

El backend se divide en dos módulos funcionales, contruidos sobre un núcleo compartido. Cada módulo respeta internamente la misma división en cuatro capas. La Figura 4 muestra los módulos y sus componentes principales, y la Tabla 20 resume la responsabilidad de cada uno.

Módulos del backend y componentes principales

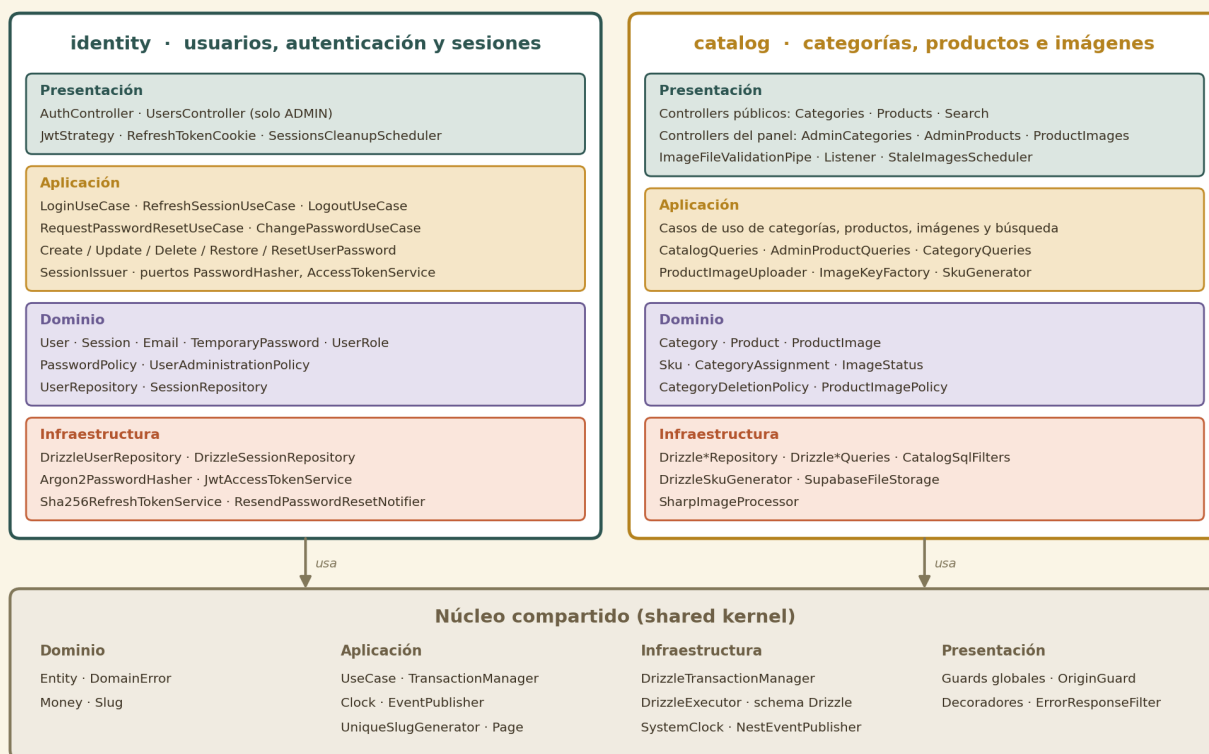


Figura 4. Módulos del backend y sus componentes principales

Tabla 20. Módulos del backend

Módulo	Responsabilidad	Relación con el resto
identity	Usuarios del panel, autenticación, sesiones y contraseñas.	Provee la autenticación y la autorización de toda la API, y la identidad del usuario que se registra en la auditoría del catálogo.
catalog	Categorías, productos, imágenes y búsquedas del catálogo público.	Usa los guards de identity para proteger las operaciones del panel y registra qué usuario creó o modificó cada producto.
Núcleo compartido	Abstracciones comunes: entidad base, errores, transacciones, fecha actual, eventos, slugs y paginación.	Todas las entidades, errores y casos de uso de ambos módulos se construyen sobre él.

8.3 Núcleo compartido

Diagrama de referencia: sección «2. Shared kernel».

El núcleo compartido evita duplicar código y garantiza que los aspectos transversales se resuelvan de la misma forma en todo el backend. Sus componentes principales son:

- **Dominio:** Entity (entidad base), la jerarquía de errores de negocio y los value objects Money, que opera con centavos enteros para evitar errores de redondeo, y Slug, para los identificadores de las URL.
- **Aplicación:** UseCase (interfaz común con un único método execute), TransactionManager, Clock (fecha actual, reemplazable en las pruebas), EventPublisher, UniqueSlugGenerator (agrega un sufijo numérico si el slug ya existe) y las estructuras de paginación Page y PageRequest.

- **Infraestructura:** DrizzleTransactionManager y DrizzleExecutor para las transacciones, SystemClock y NestEventPublisher.
- **Presentación:** los guards globales, los decoradores (@Public, @Roles, @AllowPendingPasswordChange, @CurrentUser) y ErrorResponseFilter.

Transacciones. Cuando un caso de uso necesita varias operaciones atómicas, TransactionManager abre una transacción y la guarda en un contexto de ejecución (AsyncLocalStorage); cada repositorio obtiene de allí la transacción activa, sin recibirla como parámetro. Como el almacenamiento de archivos no participa de la transacción, se aplica una estrategia de compensación: los archivos se suben antes de iniciarla y se eliminan si falla, y las eliminaciones de archivos se realizan solo después de confirmarla.

Mapeo de errores. ErrorResponseFilter convierte cualquier error en una respuesta HTTP con el código adecuado, sin exponer detalles internos. Incluye también las violaciones de restricciones de PostgreSQL, de modo que las reglas aplicadas en la base lleguen al cliente con un mensaje claro (ver Tabla 21):

Tabla 21. Correspondencia entre errores y códigos de respuesta HTTP

Error	Código HTTP
InvalidValueError y errores de validación de datos de entrada	400
AuthenticationError	401
ForbiddenActionError	403
EntityNotFoundError	404
ConflictError y errores de PostgreSQL 23505 (valor duplicado) y 23503 (referencia inválida)	409
BusinessRuleViolationError y error de PostgreSQL 23514 (CHECK o trigger de validación)	422
Cualquier otro error	500, sin exponer detalles internos

Guards globales. Se ejecutan en el siguiente orden antes de cada solicitud:

- **ThrottlerGuard:** limita la cantidad de solicitudes para prevenir abusos.
- **JwtAuthGuard:** verifica que el usuario esté autenticado, salvo en los endpoints marcados como públicos.
- **MustChangePasswordGuard:** bloquea el acceso si el usuario tiene pendiente el cambio de su contraseña temporal.
- **RolesGuard:** verifica que el rol del usuario (ADMIN o SELLER) tenga permiso para la operación.

Además, OriginGuard, que no es global, se aplica solo en los endpoints que leen la cookie del refresh token y verifica que la solicitud provenga del dominio del frontend.

8.4 Módulo identity

Gestiona los usuarios del panel, su autenticación y el control de sus sesiones. Su elemento central es la entidad User, que concentra los datos, el rol, el estado, las contraseñas y la baja lógica de cada usuario.

Diagrama de referencia: sección «3. Identity · Dominio».

Dominio. Clases principales y su responsabilidad (ver Tabla 22):

Tabla 22. Clases del dominio del módulo identity

Clase	Tipo	Responsabilidad
User	Entidad (raíz de agregado)	Único punto de acceso para modificar un usuario; sus métodos validan las reglas antes de aplicar cada cambio.
Session	Entidad	Sesión iniciada con un refresh token; es válida mientras no venza ni sea revocada.
Email	Value object	Valida el formato y normaliza el email en minúsculas.
TemporaryPassword	Value object	Clave temporal por olvido (hash y vencimiento) y período de espera entre solicitudes.
UserRole	Enumeración	Roles ADMIN y SELLER.
PasswordPolicy	Servicio de dominio	Requisitos de toda contraseña: entre 8 y 128 caracteres.
UserAdministrationPolicy	Servicio de dominio	Reglas entre usuarios: un administrador no puede cambiar su rol, desactivarse ni darse de baja, y siempre debe existir un administrador activo.
UserRepository, SessionRepository	Contratos	Acceso a usuarios y sesiones, con operaciones propias del negocio como contar administradores activos o revocar sesiones.

Diagrama de referencia: sección «4. Identity · Aplicación».

Casos de uso. Cada operación se implementa como un caso de uso independiente (ver Tabla 23):

Tabla 23. Casos de uso del módulo identity

Grupo	Casos de uso
Autenticación y sesiones	LoginUseCase, RefreshSessionUseCase, LogoutUseCase, RequestPasswordResetUseCase, ChangePasswordUseCase, ResolveAuthenticatedUserUseCase, GetCurrentUserUseCase y PurgeExpiredSessionsUseCase.
Administración de usuarios (solo ADMIN)	ListUsersUseCase, GetUserUseCase, CreateUserUseCase, UpdateUserUseCase, ResetUserPasswordUseCase, DeleteUserUseCase y RestoreUserUseCase.

Flujos principales. Los flujos de autenticación incorporan medidas de seguridad específicas:

- **Inicio de sesión:** acepta la contraseña actual o una clave temporal vigente; si se usa la temporal, pasa a ser la contraseña y se exige cambiarla. Ante email inexistente, contraseña incorrecta o usuario inactivo devuelve siempre el mismo error y ejecuta igualmente una verificación de hash, para que el tiempo de respuesta no revele qué cuentas existen.
- **Renovación de sesión:** cada refresh token se usa una sola vez: la sesión usada se revoca y se emite una nueva. Si se presenta un token ya revocado, se interpreta como un posible robo y se revocan todas las sesiones del usuario.
- **Recuperación de contraseña:** finaliza siempre sin error, exista o no el email. Si corresponde, genera una clave temporal de 30 minutos y la envía por mail, con una espera mínima de 5 minutos entre solicitudes; no invalida la contraseña actual ni cierra sesiones.

- **Cambio de contraseña:** disponible para ambos roles; verifica la contraseña actual y emite una sesión nueva después de guardar, ya que la base revoca las anteriores.
- **Validación en cada solicitud:** el rol, el estado y la obligación de cambio de contraseña se leen de la base de datos y no del token, por lo que cualquier cambio sobre el usuario tiene efecto inmediato.
- **Reseteo por un administrador:** dentro de una transacción se genera la clave temporal, se guarda y se envía por mail; si el envío falla, la operación se revierte y el usuario conserva su contraseña.

Diagrama de referencia: sección «5. Identity · Infraestructura y presentación».

Infraestructura. Cada contrato se implementa con una tecnología concreta (ver Tabla 24):

Tabla 24. Implementaciones de infraestructura del módulo identity

Contrato	Implementación	Detalle
PasswordHasher	Argon2PasswordHasher	Hash de contraseñas con Argon2id.
TemporaryPasswordGenerator	CryptoTemporaryPasswordGenerator	Claves de 12 caracteres con un generador criptográfico, sin caracteres confundibles (0, O, 1, l, I).
AccessTokenService	JwtAccessTokenService	JWT firmado con HS256, de 15 minutos, que contiene solo el identificador del usuario.
RefreshTokenService	Sha256RefreshTokenService	Tokens de 32 bytes aleatorios; en la base se guarda solo su hash SHA-256.
PasswordResetNotifier	ResendPasswordResetNotifier	Mail en texto y HTML con la clave temporal, su vencimiento y el enlace al panel.
UserRepository, SessionRepository, UserQueries	Drizzle*Repository y DrizzleUserQueries	Acceso a users y sessions con Drizzle, dentro de la transacción activa si existe.

En la capa de presentación, AuthController expone la autenticación y UsersController, restringido en su totalidad al rol ADMIN, la administración de usuarios. Completan la capa RefreshTokenCookie, que centraliza el manejo de la cookie, JwtStrategy y la tarea programada que elimina las sesiones vencidas.

8.5 Módulo catalog

Gestiona todo el contenido que se exhibe en la tienda. Su dominio se organiza en tres agregados independientes: categorías, productos e imágenes. Las imágenes forman un agregado propio porque tienen operaciones y un ciclo de procesamiento asíncrono independientes del producto.

Diagrama de referencia: sección «6. Catalog · Dominio».

Dominio. Clases principales y su responsabilidad (ver Tabla 25):

Tabla 25. Clases del dominio del módulo catalog

Clase	Tipo	Responsabilidad
Category	Entidad (raíz de agregado)	Categoría con su jerarquía; puede renombrarse, moverse (sin formar ciclos), activarse, darse de baja y reactivarse.
Product	Entidad (raíz de agregado)	Producto con precio, oferta, stock, marcas, categorías y similares; valida precios, categorías y similares en cada cambio.
ProductImage	Entidad (raíz de agregado)	Imagen con su posición, claves de archivo y estado; se relaciona con el producto solo por su identificador.
Sku, CategoryAssignment, ImageStatus	Value objects y enumeración	Código LMS-000001, asignación de categoría (principal o no) y estados PENDING, PROCESSED y FAILED.
CategoryDeletionPolicy	Servicio de dominio	Impide dar de baja una categoría con subcategorías activas o que sea la única de algún producto vigente.
ProductImagePolicy	Servicio de dominio	Controla el mínimo y el máximo de imágenes, el reordenamiento y el reacomodo de posiciones tras una eliminación.
Category-, Product- y ProductImageRepository	Contratos	Acceso a las entidades, con operaciones como obtener ancestros o imágenes pendientes.

Diagramas de referencia: secciones «7. Catalog · Aplicación · Categorías y búsqueda», «8. Catalog · Aplicación · Productos» y «9. Catalog · Aplicación · Imágenes».

Casos de uso. Se agrupan según el área que atienden (ver Tabla 26):

Tabla 26. Casos de uso del módulo catalog

Grupo	Casos de uso
Categorías	ListCategoryTreeUseCase (tienda) y List-, Get-, Create-, Update-, Delete- y RestoreCategoryUseCase (panel).
Productos: tienda	SearchCatalogUseCase, ListFeaturedProductsUseCase, ListNewProductsUseCase y GetProductDetailUseCase.
Productos: panel	ListAdminProductsUseCase, GetAdminProductUseCase, CreateProductUseCase, UpdateProductUseCase, DeleteProductUseCase y RestoreProductUseCase.
Imágenes: panel	List-, Add-, Update-, ReplaceFile-, Reorder-, Delete- y RetryProductImageProcessingUseCase.
Imágenes: procesamiento	ProcessProductImageUseCase y ReprocessStaleImagesUseCase.
Búsqueda	GlobalSearchUseCase, que busca productos y categorías a la vez.

Reglas destacadas de los casos de uso.

- **Visibilidad:** la tienda sólo muestra productos activos, no dados de baja y con al menos una imagen procesada; las marcas de oferta y novedad se informan como vigentes solo si no vencieron.
- **Catálogo:** 20 productos por página por defecto y 50 como máximo; el orden por relevancia se aplica solo cuando hay texto de búsqueda.

- **Destacados y novedades:** hasta 5 de cada uno, ordenados por posición y, a igual posición, del más reciente al más antiguo. El límite se aplica a lo que se muestra, no a la cantidad de productos marcados.
- **Productos similares:** se muestran 4; primero los cargados a mano y, si no alcanzan, se completan con productos visibles de la misma categoría principal.
- **Búsqueda global:** requiere al menos 2 caracteres y devuelve 5 resultados por grupo por defecto, con un máximo de 10.
- **Categorías:** la reactivación no se aplica en cascada a las subcategorías y se rechaza si la categoría padre está dada de baja.
- **Productos:** al editar, las categorías y similares enviados reemplazan por completo a los anteriores; la baja no modifica imágenes ni archivos, y la reactivación exige que el producto conserve al menos una categoría vigente.

Diagrama de referencia: sección «10. Catalog · Infraestructura y presentación».

Infraestructura y presentación. Implementaciones y componentes principales (ver Tabla 27):

Tabla 27. Infraestructura y presentación del módulo catalog

Contrato o componente	Implementación	Detalle
Repositorios y consultas	Drizzle*Repository, Drizzle*Queries	Acceso con Drizzle; CatalogSqlFilters centraliza visibilidad, vigencia de ofertas y búsqueda sin tildes.
SkuGenerator	DrizzleSkuGenerator	Invoca next_product_sku(), que usa la secuencia de la base y evita SKU repetidos aun ante altas simultáneas.
FileStorage	SupabaseFileStorage	Bucket privado para originales y bucket público para variantes; URL firmadas para ver originales en el panel.
ImageProcessor	SharpImageProcessor	Genera las variantes WebP con la librería Sharp.
Validación de archivos	ImageFileValidationPipe	Máximo 10 MB; verifica el tipo declarado y los primeros bytes del archivo (JPEG, PNG o WebP).
Controladores	Públicos y del panel	Categories, Products y Search son públicos; AdminCategories, AdminProducts y ProductImages, para ADMIN y SELLER.

8.6 Alta de producto y procesamiento de imágenes

Diagramas de referencia: secciones «8. Catalog · Aplicación · Productos» y «9. Catalog · Aplicación · Imágenes».

El alta de producto es la operación más compleja del sistema, porque coordina validaciones, archivos, base de datos y un procesamiento posterior. La Figura 5 resume su recorrido completo.

Alta de producto y procesamiento de imágenes



La respuesta al usuario se envía tras el paso 5; el procesamiento continúa en segundo plano.

Figura 5. Alta de producto y procesamiento de imágenes

El procesamiento de imágenes se realiza de forma asincrónica: al cargar o reemplazar una imagen, el sistema registra el archivo original, publica un evento y responde de inmediato, mientras un listener procesa la imagen en segundo plano. Así, la carga desde el panel resulta ágil incluso desde un celular o con conexiones lentas.

Almacenamiento de los archivos. Cada imagen se guarda en dos lugares:

- **Original:** en el bucket privado, con la clave products/{productId}/{imageId}.{ext}; no es accesible desde la tienda.
- **Variantes:** en el bucket público, con la clave products/{productId}/{imageId}-v{versión}-{ancho}.webp. La versión cambia en cada procesamiento, por lo que la URL de una imagen reemplazada también cambia y ninguna caché sigue mostrando la anterior.

Gestión desde el panel. Las demás operaciones sobre imágenes siguen el mismo criterio:

- **Reemplazo:** la imagen vuelve a estado pendiente y las variantes anteriores siguen publicadas hasta que termina el nuevo procesamiento, para que el producto no quede sin imagen en la tienda.
- **Reordenamiento:** se aplica dentro de una transacción, difiriendo la restricción de posición única hasta el COMMIT.
- **Eliminación:** no se permite borrar la última imagen; las posiciones se reacomodan y los archivos se eliminan solo después de confirmar la transacción.
- **Visualización:** el panel muestra la variante de 800 px si la imagen está procesada, o el original mediante una URL firmada de validez limitada mientras está pendiente o si falló.

8.7 Seguridad y manejo de la sesión

La autenticación combina dos tokens con distinta duración y forma de almacenamiento, para reducir el impacto de un eventual robo de credenciales. El access token (JWT de 15 minutos) se entrega en el cuerpo de la respuesta, el frontend lo conserva solo en memoria y lo envía en el encabezado Authorization. El refresh token se entrega en una cookie con la siguiente configuración (ver Tabla 28):

Tabla 28. Configuración de la cookie del refresh token

Atributo	Valor	Propósito
HttpOnly	Activado	No es accesible desde JavaScript, lo que la protege ante inyección de scripts (XSS).
Secure	Activado	Solo se transmite por conexiones HTTPS.
SameSite	Strict	No se envía en solicitudes originadas en otros sitios, lo que protege ante falsificación de solicitudes (CSRF).
Path	/api/v1/auth	Solo se envía a los endpoints de autenticación.
Domain	Sin definir	Solo se envía al servidor de la API y no a otros subdominios.
Nombre	__Secure-refresh_token	El prefijo obliga al navegador a rechazarla si no fue establecida por HTTPS.
Duración	7 días	Rota en cada uso, con detección de reutilización.

- **Renovación y cierre de sesión:** leen la cookie y no requieren el access token, por lo que se puede cerrar la sesión aunque este haya vencido; ambos aplican OriginGuard como defensa adicional ante CSRF.
- **CORS y dominio compartido:** la API solo acepta solicitudes del dominio del frontend, con envío de credenciales habilitado. Por la política SameSite=Strict, el sitio y la API deben compartir el dominio registrado (libreriamisueno.com.ar y api.libreriamisueno.com.ar).
- **Recarga de la página:** como el access token vive en memoria, al recargar el frontend solicita automáticamente una renovación, sin volver a pedir el inicio de sesión.
- **Identidad del autor:** el identificador del usuario que realiza cada acción se toma de la sesión autenticada y nunca del cuerpo de la solicitud, lo que impide suplantar al autor de una operación.

Límites de solicitudes. Toda la API aplica un límite general por dirección IP, y los endpoints más expuestos tienen límites específicos (ver Tabla 29):

Tabla 29. Límites de solicitudes por endpoint

Endpoint	Límite
POST /auth/login	5 solicitudes por minuto por IP
POST /auth/forgot-password	3 solicitudes cada 15 minutos por IP
POST /auth/refresh	Límite general de la API

8.8 API REST

La API sigue estas convenciones generales:

- **Versionado:** todas las rutas se exponen bajo el prefijo /api/v1, que se omite en las tablas por legibilidad (por ejemplo, /auth/login corresponde a https://api.libreriamisueno.com.ar/api/v1/auth/login).
- **Rutas públicas y del panel:** las rutas que comienzan con /admin requieren autenticación; las demás del catálogo y la búsqueda son públicas.
- **Bajas lógicas:** el método DELETE realiza una baja lógica, ya que para el cliente el recurso deja de estar disponible; la reactivación se hace con el endpoint restore.
- **Identificadores:** los endpoints públicos identifican al producto por su slug, para generar URL legibles y favorables al posicionamiento; los del panel, por su identificador interno.

Autenticación. Los endpoints de cambio de contraseña y de consulta del usuario actual son los únicos habilitados mientras hay un cambio de contraseña pendiente (ver Tabla 30).

Tabla 30. Endpoints de autenticación

Método	Ruta	Acceso	Caso de uso
POST	/auth/login	Público	LoginUseCase
POST	/auth/refresh	Cookie y verificación de origen	RefreshSessionUseCase
POST	/auth/logout	Cookie y verificación de origen	LogoutUseCase
POST	/auth/forgot-password	Público	RequestPasswordResetUseCase
POST	/auth/change-password	Autenticado (aun con cambio pendiente)	ChangePasswordUseCase
GET	/auth/me	Autenticado (aun con cambio pendiente)	GetCurrentUserUseCase

Usuarios. El reseteo de contraseña no recibe datos: el sistema genera la clave temporal y la envía por mail (ver Tabla 31).

Tabla 31. Endpoints de usuarios

Método	Ruta	Acceso	Caso de uso
GET	/admin/users	ADMIN	ListUsersUseCase
GET	/admin/users/:id	ADMIN	GetUserUseCase
POST	/admin/users	ADMIN	CreateUserUseCase
PATCH	/admin/users/:id	ADMIN	UpdateUserUseCase
POST	/admin/users/:id/reset-password	ADMIN	ResetUserPasswordUseCase
DELETE	/admin/users/:id	ADMIN	DeleteUserUseCase
POST	/admin/users/:id/restore	ADMIN	RestoreUserUseCase

Categorías. Endpoints públicos y del panel para la consulta y gestión de categorías (ver Tabla 32).

Tabla 32. Endpoints de categorías

Método	Ruta	Acceso	Caso de uso
GET	/categories	Público	ListCategoryTreeUseCase
GET	/admin/categories	ADMIN, SELLER	ListAdminCategoriesUseCase
GET	/admin/categories/:id	ADMIN, SELLER	GetAdminCategoryUseCase
POST	/admin/categories	ADMIN, SELLER	CreateCategoryUseCase
PATCH	/admin/categories/:id	ADMIN, SELLER	UpdateCategoryUseCase
DELETE	/admin/categories/:id	ADMIN, SELLER	DeleteCategoryUseCase
POST	/admin/categories/:id/restore	ADMIN, SELLER	RestoreCategoryUseCase

Productos. El alta recibe los datos y entre 1 y 5 imágenes en una única solicitud multipart/form-data (ver Tabla 33).

Tabla 33. Endpoints de productos

Método	Ruta	Acceso	Caso de uso
GET	/products	Público	SearchCatalogUseCase
GET	/products/featured	Público	ListFeaturedProductsUseCase
GET	/products/new	Público	ListNewProductsUseCase
GET	/products/:slug	Público	GetProductDetailUseCase
GET	/admin/products	ADMIN, SELLER	ListAdminProductsUseCase
GET	/admin/products/:id	ADMIN, SELLER	GetAdminProductUseCase
POST	/admin/products	ADMIN, SELLER	CreateProductUseCase
PATCH	/admin/products/:id	ADMIN, SELLER	UpdateProductUseCase
DELETE	/admin/products/:id	ADMIN, SELLER	DeleteProductUseCase
POST	/admin/products/:id/restore	ADMIN, SELLER	RestoreProductUseCase

Imágenes de producto y búsqueda. El reemplazo usa PUT porque sustituye el archivo completo; la edición usa PATCH porque modifica solo algunos datos, como el texto alternativo (ver Tabla 34).

Tabla 34. Endpoints de imágenes de producto y búsqueda

Método	Ruta	Acceso	Caso de uso
GET	/admin/products/:productId/images	ADMIN, SELLER	ListProductImagesUseCase
POST	/admin/products/:productId/images	ADMIN, SELLER	AddProductImageUseCase
PATCH	/admin/products/:productId/images/order	ADMIN, SELLER	ReorderProductImagesUseCase

Método	Ruta	Acceso	Caso de uso
PATCH	/admin/products/:productId/images/:imageId	ADMIN, SELLER	UpdateProductImageUseCase
PUT	/admin/products/:productId/images/:imageId/file	ADMIN, SELLER	ReplaceProductImageFileUseCase
POST	/admin/products/:productId/images/:imageId/retry	ADMIN, SELLER	RetryProductImageProcessingUseCase
DELETE	/admin/products/:productId/images/:imageId	ADMIN, SELLER	DeleteProductImageUseCase
GET	/search	Público	GlobalSearchUseCase

8.9 Persistencia con Drizzle

Diagrama de referencia: sección «11. Schema Drizzle».

El schema de Drizzle es el reflejo en TypeScript del modelo de datos de la sección 7. Se ubica en la infraestructura del núcleo compartido y solo lo utilizan repositorios, consultas y mappers; dominio y aplicación no lo conocen.

- **Declaración:** las tablas se declaran dentro del schema libreria; cada columna conserva su nombre snake_case de la base, mientras que el código usa camelCase.
- **Tipos especiales:** citext se define como tipo personalizado, y las columnas numeric se leen como texto para evitar errores de redondeo; los mappers las convierten al value object Money.
- **Mappers:** traducen los registros en entidades y viceversa; son el único punto de contacto entre ambos modelos, por lo que un cambio en las tablas no afecta las reglas de negocio.
- **Migraciones:** drizzle-kit genera las migraciones de tablas, claves e índices convencionales; funciones, triggers, la secuencia del SKU y el índice de búsqueda se agregan con una migración SQL manual (drizzle-kit generate --custom) dentro del mismo historial.
- **Evolución:** las futuras tablas de pedidos y pagos se incorporarán como nuevas declaraciones y migraciones, sin modificar las existentes.

8.10 Validación de las reglas de negocio en capas

Las reglas más importantes no dependen de un único punto de control. La presentación rechaza temprano las solicitudes inválidas con errores claros; la aplicación y el dominio son la fuente principal de las reglas; y la base de datos actúa como última barrera. La Tabla 35 muestra dónde se aplica cada regla.

Tabla 35. Validación de las reglas de negocio por capa

Regla	Presentación	Aplicación / Dominio	Base de datos
Solo ADMIN gestiona usuarios	RolesGuard; UsersController restringido a ADMIN	No aplica	No aplica
Email único entre usuarios vigentes	Formato en el DTO	Email y casos de uso de alta, edición y reactivación	chk_users_email_format y uq_users_email_not_deleted

Regla	Presentación	Aplicación / Dominio	Base de datos
Contraseña de 8 a 128 caracteres	Validación del DTO	PasswordPolicy	No aplica
Cambio obligatorio tras clave temporal	MustChangePasswordGuard	User.assignTemporaryPassword y consumeTemporaryPassword	No aplica
Siempre existe un administrador activo	No aplica	UserAdministrationPolicy	Trigger prevent_last_admin_removal
Refresh token rotativo con detección de reuso	Cookie HttpOnly y OriginGuard	RefreshSessionUseCase	uq_sessions_refresh_token_hash
SKU generado e inmutable	Los DTOs no lo aceptan	SkuGenerator; Product no permite modificarlo	Secuencia, CHECK de formato y trigger
Precio de oferta menor al regular	Validación del DTO	Product.startSale y changePrice	chk_products_sale_price_range
Al menos una categoría y una principal	Validación del DTO	CategoryAssignment y Product.assignCategories	Constraint trigger diferido e índice único parcial
Entre 1 y 5 imágenes por producto	ImageFileValidationPipe en el alta	ProductImagePolicy	Constraint trigger diferido, CHECK y UNIQUE diferible
Imágenes JPEG, PNG o WebP de hasta 10 MB	ImageFileValidationPipe (tipo declarado y contenido real)	ImageSettings	chk_product_images_mime_type
Visibilidad en la tienda	No aplica	CatalogQueries mediante CatalogSqlFilters	Índices parciales
No reactivar un producto sin categorías vigentes	No aplica	RestoreProductUseCase	No aplica
Bajas lógicas reversibles	Sin endpoints de borrado definitivo; endpoints restore	Métodos softDelete y restore	deleted_at e índices únicos parciales

8.11 Decisiones de diseño fundamentadas

La Tabla 36 resume las decisiones de diseño del backend, la alternativa considerada y el fundamento de cada elección.

Tabla 36. Decisiones de diseño del backend

Decisión	Alternativa descartada	Fundamento
Clean Architecture en cuatro capas	Controladores y servicios acoplados al framework y al ORM	Aísla las reglas de negocio, permite probar los casos de uso sin infraestructura y reemplazar proveedores sin modificar la lógica.

Decisión	Alternativa descartada	Fundamento
Contratos como clases abstractas	Interfaces con identificadores de inyección propios	NestJS las usa directamente para la inyección y TypeScript verifica que cada implementación cumpla el contrato.
Puertos de consulta para lecturas	Reconstruir entidades en cada listado	Mejor rendimiento y respuestas con el formato exacto de cada pantalla.
Transacción en AsyncLocalStorage	Pasar la transacción como parámetro	Los casos de uso no la transmiten entre repositorios y el dominio no conoce su existencia.
Procesamiento de imágenes asíncrono	Procesar dentro de la misma solicitud	Respuesta inmediata al panel, con reintento manual y reprocesamiento programado ante fallas.
Compensación de archivos	Tratar el almacenamiento como parte de la transacción	El almacenamiento no es transaccional: se sube antes, se elimina si falla y se borra solo tras confirmar.
Rol y estado leídos de la base en cada solicitud	Incluirlos en el JWT	Un cambio de rol, estado o contraseña tiene efecto inmediato, sin esperar el vencimiento del token.
Access token en memoria y refresh token en cookie HttpOnly	Ambos tokens en localStorage	Un script inyectado no puede leer los tokens para usarlos desde otro dispositivo.
Refresh token con hash SHA-256	Argon2id, como las contraseñas	El token es aleatorio y de alta entropía: un hash lento no agrega seguridad y sí costo.
API versionada bajo /api/v1	Rutas sin versión	Permite introducir cambios futuros sin afectar a los clientes existentes.
DELETE para la baja lógica y restore para reactivar	Endpoints específicos de baja	Para el cliente el recurso deja de estar disponible; la semántica HTTP se mantiene simple.
Límite de 5 destacados y novedades mostrados	Limitar la cantidad de productos marcados	Se pueden marcar todos los productos deseados y controlar cuáles aparecen primero por su posición.
Resend para el envío de mails	Otros proveedores de correo transaccional	SDK oficial para Node.js; su plan gratuito (3.000 mails por mes, 100 por día y un dominio; Resend, s.f.) cubre el bajo volumen previsto.

9. Frontend

Esta sección describe el diseño del frontend: los criterios de arquitectura que lo organizan, el mapa de rutas de la aplicación, los módulos que la componen y sus responsabilidades, las pantallas que resultan de combinarlos, la estructura de carpetas, la integración con la API REST descrita en la sección 8 y las decisiones de diseño que lo fundamentan. El frontend se desarrolla con Next.js 14 (App Router), TypeScript, Tailwind CSS y shadcn/ui, se aloja en Vercel y es responsable exclusivamente de la interfaz: la tienda pública y el panel de administración.

Al igual que el backend, el frontend se organiza en módulos funcionales con una regla de dependencias explícita, de modo que la lógica del lado del cliente —como el cálculo de los totales del carrito, el armado del mensaje de WhatsApp o las validaciones de los formularios— quede separada de los componentes visuales y del acceso a la API. El diseño visual toma como guía un prototipo navegable de la tienda, utilizado solo como referencia.

Documento de referencia. El detalle completo de módulos, rutas y pantallas, junto con el diagrama de dependencias entre módulos, está disponible en el repositorio del proyecto (ver Anexo 1.2), donde GitHub lo muestra directamente como diagrama.

9.1 Criterios de arquitectura

La organización del frontend responde a los siguientes criterios:

- **Organización por módulo funcional:** cada módulo agrupa todo lo necesario para una funcionalidad: tipos, reglas, llamadas a la API, estado y componentes. Las rutas de la carpeta `app/` solo combinan módulos para formar cada pantalla y no contienen lógica propia.
- **Capas dentro de cada módulo:** cada módulo se divide en capas con la misma regla de dependencias que el backend: las capas externas dependen de las internas y nunca al revés (ver Tabla 37). De esta forma, las reglas del dominio pueden probarse con tests unitarios simples, sin React ni llamadas a la API.
- **Tienda pública renderizada en el servidor:** las páginas públicas se generan en el servidor mediante SSR e ISR, lo que favorece el posicionamiento en buscadores y la velocidad de carga en celulares (RNF-02). Se revalidan cada 60 segundos, por lo que un cambio realizado en el panel se refleja en la tienda en, como máximo, un minuto.
- **Panel renderizado en el cliente:** el refresh token vive en una cookie `HttpOnly` del dominio de la API (`api.libreriamisueno.com.ar`), por lo que el servidor de Next.js no la recibe. El panel se protege del lado del cliente: al cargar solicita un access token nuevo y, si no lo obtiene, redirige al login. La seguridad real la aplica la API en cada endpoint (ver 8.7).

Tabla 37. Capas dentro de cada módulo del frontend

Capa	Contenido
<code>domain/</code>	Tipos y funciones puras, como el cálculo de totales, el armado del mensaje de WhatsApp o las reglas de stock. No utiliza React ni realiza llamadas a la API, por lo que se prueba con tests unitarios simples.
<code>application/</code>	Hooks, stores y acciones que orquestan cada caso de uso de la interfaz, por ejemplo <code>useAddToCart</code> o <code>useLogin</code> .
<code>infrastructure/</code>	Llamadas a la API y conversión de las respuestas del backend a los tipos del dominio.
<code>ui/</code>	Componentes visuales. Solo utilizan las capas <code>application</code> y <code>domain</code> , y nunca llaman a la API directamente.

Las librerías que complementan el stack, junto con el motivo de su elección, se resumen en la Tabla 38.

Tabla 38. Librerías del frontend

Uso	Librería	Motivo
Componentes de interfaz	shadcn/ui y Tailwind CSS	Componentes accesibles que se incorporan al propio proyecto y pueden adaptarse al diseño, con estilos utilitarios que facilitan el diseño adaptable.
Formularios y validación	React Hook Form + Zod	Formularios eficientes y esquemas de validación tipados que replican las reglas del backend, como la longitud de la contraseña o el precio de oferta menor al precio regular.
Datos del panel (listados, mutaciones y caché)	TanStack Query	Caché de consultas, estados de carga y error, y actualización de los listados después de cada alta, edición o baja.
Carrito	Zustand con persistencia en localStorage	Estado global liviano que se conserva entre visitas sin necesidad de tablas ni endpoints.
Ordenamiento de imágenes	dnd-kit	Arrastre compatible con mouse, pantallas táctiles y teclado.
Íconos	lucide-react	Íconos livianos, utilizados también para representar las categorías en la tienda.

9.2 Mapa de rutas

La Tabla 39 presenta las rutas de la aplicación, agrupadas por área, junto con la forma en que se genera cada pantalla y el nivel de acceso requerido. Las rutas públicas usan nombres en español, pensados para el posicionamiento en buscadores, y el filtro de categoría del catálogo utiliza el slug de la categoría (por ejemplo, /productos?categoria=cuadernos).

Tabla 39. Mapa de rutas del frontend

Área	Ruta	Pantalla	Render	Acceso
Tienda	/	Inicio	ISR	Público
Tienda	/productos	Catálogo (?q=&categoria=&pagina=)	SSR	Público
Tienda	/productos/[slug]	Ficha de producto	ISR	Público
Tienda	/sobre-nosotros	Sobre nosotros	Estático	Público
Tienda	Panel lateral global	Carrito	Cliente	Público
Acceso	/admin/login	Inicio de sesión	Cliente	Público
Acceso	/admin/olvide-mi-contrasena	Pedido de clave temporal	Cliente	Público
Acceso	/admin/cambiar-contrasena	Cambio de contraseña (obligatorio o voluntario)	Cliente	Autenticado
Panel	/admin	Redirige a /admin/productos	Cliente	Autenticado
Panel	/admin/productos	Listado de productos	Cliente	ADMIN, SELLER

Área	Ruta	Pantalla	Render	Acceso
Panel	/admin/productos/nuevo	Alta de producto	Cliente	ADMIN, SELLER
Panel	/admin/productos/[id]	Edición de producto	Cliente	ADMIN, SELLER
Panel	/admin/categorias	Listado de categorías	Cliente	ADMIN, SELLER
Panel	/admin/categorias/nueva	Alta de categoría	Cliente	ADMIN, SELLER
Panel	/admin/categorias/[id]	Edición de categoría	Cliente	ADMIN, SELLER
Panel	/admin/usuarios	Listado de usuarios	Cliente	ADMIN
Panel	/admin/usuarios/nuevo	Alta de usuario	Cliente	ADMIN
Panel	/admin/usuarios/[id]	Edición de usuario	Cliente	ADMIN
SEO	/sitemap.xml, /robots.txt	Mapa del sitio y reglas para buscadores	Servidor	Público

La columna Render indica cómo se genera cada pantalla: ISR, en el servidor y con revalidación periódica; SSR, en el servidor en cada solicitud, porque el resultado depende de la búsqueda y de los filtros; estático, una única vez al compilar la aplicación; y cliente, directamente en el navegador. El nivel de acceso coincide con el que la API exige en los endpoints que utiliza cada pantalla (ver 8.8).

9.3 Módulos del sistema

Diagrama de referencia: sección «3. Módulos».

El frontend se compone de doce módulos, agrupados en tres áreas: los de la tienda pública, los del panel de administración y los transversales, que prestan servicios al resto. La Figura 6 muestra los módulos y las dependencias entre ellos, y la Tabla 40 resume la responsabilidad de cada uno.

Módulos del frontend y sus dependencias

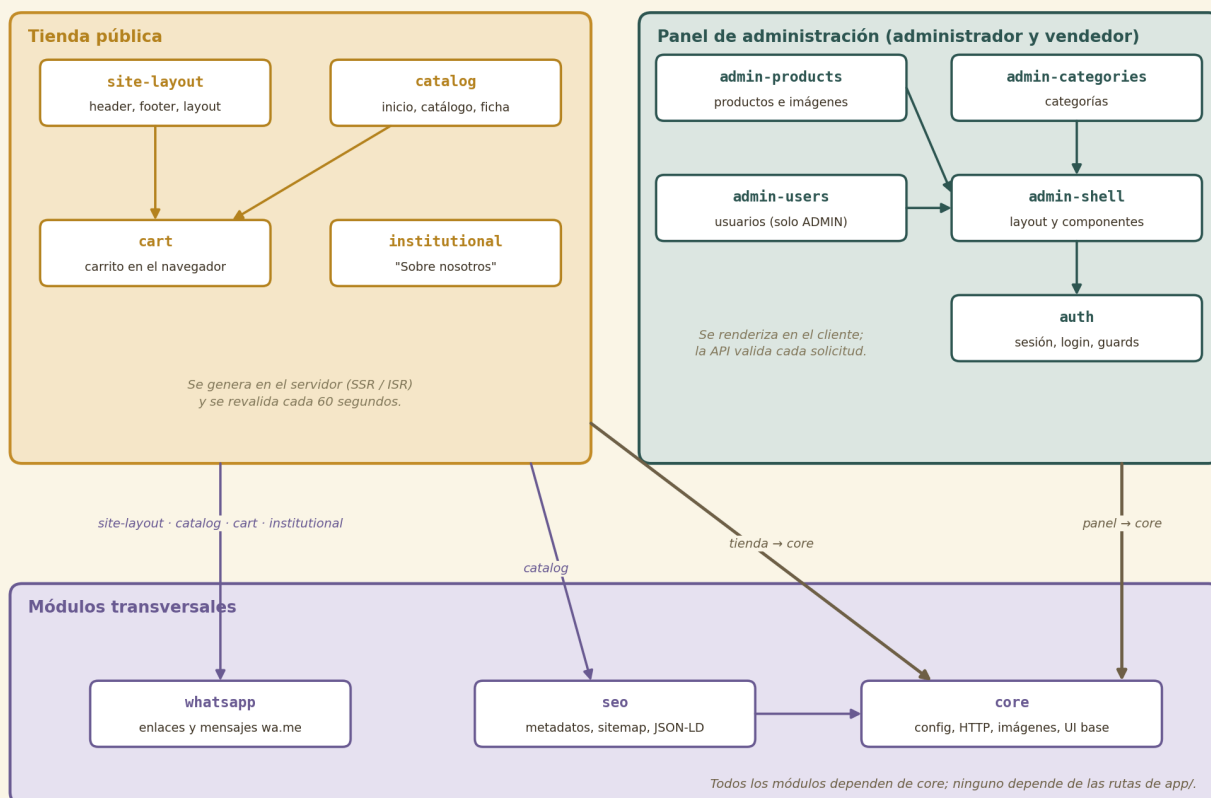


Figura 6. Módulos del frontend y sus dependencias

Tabla 40. Módulos del frontend

Módulo	Área	Responsabilidad
core	Transversal	Configuración del sitio, cliente HTTP, formateadores, loader de imágenes, componentes base y páginas de sistema.
whatsapp	Transversal	Armado de los enlaces wa.me y de los mensajes de consulta y de pedido.
seo	Transversal	Metadatos, datos estructurados, mapa del sitio, reglas para buscadores y URL canónicas.
site-layout	Tienda pública	Header, footer y estructura común de las páginas públicas.
catalog	Tienda pública	Inicio, catálogo con buscador y filtros, y ficha de producto.
cart	Tienda pública	Carrito guardado en el navegador y reserva del pedido por WhatsApp.
institutional	Tienda pública	Página "Sobre nosotros" y su resumen en el inicio.
auth	Panel	Sesión, arranque del panel, guards de ruta y pantallas de login y contraseñas.
admin-shell	Panel	Estructura del panel y componentes compartidos por listados y formularios.
admin-products	Panel	Listado y formulario de productos, y gestión de sus imágenes.

Módulo	Área	Responsabilidad
admin-categories	Panel	Listado y formulario de categorías.
admin-users	Panel (solo ADMIN)	Listado y formulario de usuarios, y envío de claves temporales.

9.4 Módulos transversales

Los módulos transversales concentran los servicios que utiliza el resto de la aplicación, de modo que cada aspecto común se resuelva en un único lugar.

Módulo core. Es la base compartida por todos los módulos; sus componentes se detallan en la Tabla 41.

Tabla 41. Componentes del módulo core

Parte	Contenido
Configuración del sitio (site-config)	Nombre, número de WhatsApp, Instagram, dirección, horarios y mail de contacto, tomados de variables de entorno públicas. Los utilizan el header, el footer, "Sobre nosotros" y los enlaces de WhatsApp, por lo que existe un único lugar para modificarlos.
Cliente HTTP	Encapsula las llamadas a la API (/api/v1) y convierte sus errores (400, 401, 403, 404, 409 y 422) en un ApiError con mensaje en español. En el panel agrega el encabezado Authorization y, ante un 401, solicita un access token nuevo una sola vez, aunque fallen varias solicitudes a la vez, y las reintenta; si la renovación también falla, cierra la sesión.
Formateadores	Precios en pesos argentinos (Intl.NumberFormat con configuración es-AR y moneda ARS) y fechas.
Imágenes	Loader propio de next/image que elige la variante WebP ya generada por el backend (400, 800 o 1200 px) según el ancho requerido, sin pasar por la optimización de imágenes de Vercel, que tiene un cupo limitado en el plan gratuito.
Componentes base	De shadcn/ui: botón, campo de texto, selector, diálogo, panel lateral, tabla, notificación y esqueleto de carga. Propios: Pagination, SearchInput con debounce, EmptyState y PriceTag, que muestra el precio anterior tachado si hay oferta.
Páginas de sistema	Páginas globales de recurso no encontrado (not-found), de error y de carga.

La renovación única del access token ante varios errores 401 simultáneos es necesaria porque el refresh token es rotativo: si dos solicitudes lo renovar en paralelo, la segunda presentaría un token ya utilizado y la API lo interpretaría como una reutilización, revocando todas las sesiones del usuario (ver 8.4).

Módulo whatsapp. Reúne las funciones puras que arman los enlaces wa.me con el número definido en la configuración del sitio. La Tabla 42 presenta los mensajes que genera el sistema.

Tabla 42. Mensajes de WhatsApp generados por el sistema

Enlace	Mensaje
Consulta general	"¡Hola Librería Mi Sueño! Quería hacer una consulta."
Consulta por producto	"¡Hola! Quería consultar por: {título} ({precio}). ¿Está disponible?", seguido del enlace a la ficha del producto.

Enlace	Mensaje
Pedido del carrito	Encabezado, una línea por producto con el formato "• {cantidad}× {título} — {subtotal}", total y pregunta por el retiro en el local, igual a lo que muestra el carrito.

Ejemplo de mensaje de pedido. El mensaje que genera el carrito tiene el siguiente formato:

```
¡Hola Librería Mi Sueño! 🌻 Quiero reservar este pedido:
```

- 2× Cuaderno Rivadavia A4 rayado — \$ 9.800
- 1× Caja de lápices de colores x24 — \$ 6.500

```
Total: $ 16.300
```

```
¿Está disponible para pasar a buscarlo por el local?
```

Los productos en oferta se envían con el precio de oferta vigente. Después de enviar el pedido, el carrito no se vacía automáticamente, ya que el cliente puede no completar el envío en WhatsApp; en su lugar, ofrece la opción "Vaciar carrito".

Módulo seo. Resuelve el posicionamiento en buscadores de la tienda pública (RNF-02); sus componentes se detallan en la Tabla 43.

Tabla 43. Componentes del módulo seo

Parte	Contenido
Metadatos	Título, descripción y datos Open Graph por página; en la ficha se usan el título, la descripción y la imagen del producto, para que el enlace se vea correctamente al compartirse.
Datos estructurados	JSON-LD de tipo Product (precio y disponibilidad) en la ficha y de tipo LocalBusiness (dirección, horarios y teléfono) en el inicio y en "Sobre nosotros".
Mapa del sitio y robots	sitemap.xml generado a partir de la API, con los productos y categorías visibles, y robots.txt que excluye las rutas del panel (/admin).
URL canónica	El catálogo filtrado declara su URL canónica para no generar contenido duplicado por las combinaciones de filtros.

9.5 Módulos de la tienda pública

La tienda pública se compone de cuatro módulos que cubren los requerimientos funcionales RF-01 a RF-06.

Módulo site-layout. Define la estructura común de todas las páginas públicas (ver Tabla 44).

Tabla 44. Componentes del módulo site-layout

Parte	Contenido
Header	Logo y nombre, navegación (Inicio, Productos y Sobre nosotros), botón de WhatsApp, botón del carrito con contador y menú desplegable en celulares. Permanece fijo al desplazarse y marca la sección activa.
Footer	Descripción del local, redes (Instagram y WhatsApp), enlaces de la tienda (catálogo y categorías principales, obtenidas de la API con caché), servicios (impresiones, anillado y RapiPago), contacto, dirección y horarios.

Parte	Contenido
Layout público	Monta el header, el footer y el panel del carrito una única vez para todas las páginas públicas, lo que garantiza que sean iguales en todo el sitio.

Módulo catalog. Reúne todo lo que muestra productos y categorías al cliente. La Tabla 45 detalla sus componentes, agrupados según la pantalla en la que se utilizan.

Tabla 45. Componentes del módulo catalog

Pantalla	Componente	Contenido
Compartido	ProductCard	Imagen principal, categoría, título, precio (con el precio anterior si hay oferta), marcas de destacado, novedad y oferta, y botón "Agregar", que suma una unidad y abre el carrito. Toda la tarjeta enlaza a la ficha.
Inicio	HeroSection	Título, buscador (redirige a /productos?q=), botón "Ver catálogo" y botón de WhatsApp.
Inicio	ServicesStrip	Franja estática con los servicios del local: impresiones, anillado, Rapipago y retiro en el local.
Inicio	FeaturedSection y NewArrivalsSection	Hasta 5 destacados y 5 novedades, en el orden definido por su posición, con acceso al catálogo filtrado. Se muestran en una grilla de 5 columnas en escritorio, de 3 en tablet y en una fila con desplazamiento horizontal en celulares.
Inicio	CategoriesSection	Accesos a las categorías que enlazan al catálogo filtrado. El ícono de cada una se obtiene de un mapa local que asocia el slug con un ícono, con un ícono genérico para las categorías no incluidas; como el slug es inmutable, la asociación no se pierde si la categoría se renombra.
Catálogo	CatalogPage	Título, buscador, filtro de categorías (barra lateral en escritorio y opciones horizontales en celulares), cantidad de resultados, grilla de 20 productos, paginación y estado "Sin resultados".
Catálogo	Estado en la URL	La búsqueda, la categoría y la página se guardan en la URL: el enlace puede compartirse, el botón "atrás" del navegador funciona y los buscadores indexan las páginas de cada categoría.
Ficha	ProductGallery	Imagen grande y miniaturas; al tocar una miniatura, pasa a la vista grande. En celulares se desliza deslizando el dedo.
Ficha	ProductInfo	Breadcrumb (Inicio › categoría principal › producto), categoría, título, precio, stock ("¡Últimas N unidades!" si quedan 6 o menos y "Sin stock" si no hay), descripción, selector de cantidad limitado al stock, y botones "Agregar al carrito" y "Consultar por WhatsApp".
Ficha	RelatedProducts	Productos similares que devuelve la API: primero los cargados manualmente y, si no alcanzan, productos de la misma categoría principal (ver 8.5).
Acceso a datos	Capa infrastructure	Llamadas públicas con revalidación de 60 segundos: búsqueda, destacados, novedades, detalle por slug y árbol de categorías.

Módulo cart. Implementa el carrito del lado del cliente, sin tablas en la base de datos (ver 7.1):

- **Dominio:** cada ítem (CartItem) guarda el identificador, el slug, el título, el precio unitario, la imagen, la cantidad y el stock del producto, y un conjunto de funciones puras permite agregar, sumar, restar y quitar unidades, calcular subtotales y el total, y limitar la cantidad al stock disponible.
- **Persistencia:** un store de Zustand guardado en localStorage con una clave versionada (lms-cart-v1) conserva una copia de los datos de cada producto, lo que permite mostrar el carrito sin consultar la API. La versión de la clave permite modificar en el futuro la estructura guardada sin afectar a los carritos existentes.
- **Actualización de datos:** el carrito no consulta la API por su cuenta. Cuando el cliente abre la ficha de un producto que ya está en el carrito, se actualizan su precio, su oferta y su stock con los datos recién obtenidos; si el stock bajó, se ajusta la cantidad y se informa al cliente.
- **Aviso de confirmación:** el carrito muestra el aviso "Precios y disponibilidad sujetos a confirmación por WhatsApp", porque los datos guardados pueden haber cambiado. La confirmación final la realiza el vendedor al responder el pedido.
- **Panel del carrito (CartDrawer):** panel lateral con el listado de productos (imagen, título, precio unitario, selector de cantidad, subtotal y opción para quitar), el total y el botón "Reservar por WhatsApp". Incluye un estado vacío con acceso al catálogo, se cierra con la tecla Esc o tocando fuera de él y, por accesibilidad, mantiene el foco dentro mientras está abierto. El header muestra un contador con la cantidad total de unidades.

Módulo institucional. Resuelve la página "Sobre nosotros" (RF-06):

- **AboutTeaser:** bloque resumido de "Sobre nosotros" en el inicio, con un botón a la página completa.
- **AboutPage:** título, descripción, botones "Escribinos" (WhatsApp) y "Ver productos", tres tarjetas con los valores del local (atención cercana, servicios al instante, y reserva y retiro) y una sección de visita con dirección, horarios y botón de WhatsApp.
- **Textos:** se guardan en un archivo de contenido dentro del módulo, lo que permite editarlos sin modificar los componentes.

9.6 Módulos del panel de administración

El panel se compone de cinco módulos que cubren los requerimientos funcionales RF-07 a RF-12. Todas sus pantallas se generan en el cliente y consumen los endpoints /admin de la API, que validan en cada solicitud la sesión y el rol del usuario.

Módulo auth. Gestiona la sesión del panel y las pantallas de acceso:

- **Sesión:** se guarda en memoria el access token, los datos del usuario (identificador, nombre, email, rol y obligación de cambiar la contraseña) y el estado de la sesión (cargando, autenticado o anónimo). El access token nunca se guarda en localStorage.
- **Arranque del panel:** al ingresar a cualquier ruta /admin se solicita una renovación de sesión (POST /auth/refresh); la cookie viaja automáticamente con la opción credentials: 'include', y mientras tanto se muestra un indicador de carga.

Los guards de ruta, que controlan el acceso a cada pantalla, se describen en la Tabla 46, y las pantallas y acciones del módulo, en la Tabla 47. Los guards replican en la interfaz los controles que la API aplica mediante MustChangePasswordGuard y RolesGuard (ver 8.3): su función es orientar al usuario, no reemplazar esos controles.

Tabla 46. Guards de ruta del panel

Guard	Comportamiento
RequireAuth	Sin sesión iniciada, redirige a /admin/login e indica la pantalla a la que se quería acceder, para volver a ella después del inicio de sesión.

Guard	Comportamiento
RequirePasswordChange	Si el usuario debe cambiar su contraseña, lo redirige a /admin/cambiar-contrasena desde cualquier otra ruta.
RequireRole	Si el rol no tiene acceso a la pantalla, por ejemplo un vendedor que ingresa a /admin/usuarios, muestra el mensaje "Sin permisos".

Tabla 47. Pantallas y acciones del módulo auth

Pantalla o acción	Contenido
LoginPage	Email, contraseña (con opción para mostrarla u ocultarla), botón "Iniciar sesión" y enlace "Olvidé mi contraseña". Ante credenciales inválidas muestra un mensaje genérico, y ante demasiados intentos (429), un aviso específico.
ForgotPasswordPage	Campo de email. Siempre muestra el mismo mensaje ("Si el email existe, te enviamos una contraseña temporal"), para no revelar qué cuentas existen.
ChangePasswordPage	Contraseña actual (o la temporal), nueva contraseña y confirmación, con las reglas visibles: entre 8 y 128 caracteres y distinta de la actual. En modo obligatorio no permite salir de la pantalla, salvo cerrando la sesión.
Cierre de sesión	Invoca POST /auth/logout, limpia la sesión y vuelve a la pantalla de inicio de sesión.

Módulo admin-shell. Define la estructura del panel: una barra lateral con Productos, Categorías y Usuarios —esta última opción visible solo para el administrador—, que en celulares se colapsa en un menú, y una barra superior con el nombre, el rol y las opciones "Cambiar contraseña" y "Cerrar sesión". Además, provee los componentes compartidos por los listados y formularios (ver Tabla 48).

Tabla 48. Componentes compartidos del panel

Componente	Contenido
DataTable	Tabla con columnas configurables, que en celulares se transforma en tarjetas.
ListToolbar	Buscador con debounce, filtro de estado (Activos, Inactivos, Dados de baja y Todos) y botón "Agregar".
Paginación y ordenamiento	Navegación entre páginas de resultados y ordenamiento por columna.
ConfirmDialog	Diálogo de confirmación para las bajas.
Notificaciones	Mensajes breves de éxito y de error después de cada operación.
StatusBadge	Indicador visual del estado de cada registro: activo, inactivo o dado de baja.
FormActions	Botones "Guardar" y "Cancelar", con aviso si se intenta salir con cambios sin guardar.

Los errores de conflicto (409) y de reglas de negocio (422) que devuelve la API se muestran como mensaje del formulario o del campo afectado, por ejemplo, "Ya existe una categoría con ese nombre".

Módulo admin-products. Resuelve la gestión de productos y de sus imágenes (RF-09 y RF-10):

- **Listado:** muestra miniatura, SKU, título, categoría principal, precio (y precio de oferta), stock, marcas de destacado, novedad y oferta, estado y fecha de modificación, con buscador por título o SKU, filtros por categoría y estado,

ordenamiento por columna y paginación. Cada fila ofrece "Editar" y "Dar de baja", con confirmación; en los productos dados de baja, la acción disponible es "Reactivar".

El alta y la edición comparten un mismo formulario, cuyos campos se describen en la Tabla 49.

Tabla 49. Campos del formulario de producto

Campo	Comportamiento
Datos	Título, descripción, precio y stock. El producto no tiene subtítulo: se utiliza la descripción.
SKU	Lo genera el sistema al crear el producto. No aparece en el alta y, en la edición, se muestra como dato de solo lectura.
Oferta	Una casilla "En oferta" habilita el precio de oferta, que debe ser menor al precio regular, y un vencimiento opcional.
Marcas	Destacado, novedad con vencimiento opcional ("nuevo hasta") y posición, que define el orden en el inicio.
Categorías	Selector múltiple con elección de la categoría principal.
Productos similares	Selector con búsqueda que excluye al propio producto.
Estado	Selector con las opciones "Activo" (visible en la tienda) e "Inactivo" (oculto). La baja y la reactivación son acciones del listado, no opciones de este selector.

Gestor de imágenes (ImageManager). Administra las imágenes de cada producto:

- **Carga:** desde la computadora o desde la galería o la cámara del celular, en formatos JPEG, PNG o WebP, de 1 a 5 imágenes, con validación de tipo y tamaño (hasta 10 MB) antes de subirlas.
- **Orden y accesibilidad:** vista previa inmediata y reordenamiento por arrastre, con botones para subir y bajar cada imagen como alternativa accesible. La primera imagen es la principal, y cada una admite un texto alternativo.
- **Alta y edición:** en el alta, las imágenes se envían junto con el producto en una única solicitud multipart; en la edición, cada acción utiliza su propio endpoint (agregar, reemplazar, reordenar, editar el texto alternativo y eliminar), y no se permite eliminar la última imagen.
- **Estado del procesamiento:** muestra el estado de cada imagen (Procesando, Lista o Error) y ofrece "Reintentar" en las que fallaron; mientras haya imágenes en proceso, consulta su estado cada pocos segundos (ver 8.6).

Módulo admin-categories. Resuelve la gestión de categorías (RF-11):

- **Listado:** nombre (con sangría según el nivel en la jerarquía), categoría padre, cantidad de productos, posición y estado, con buscador, filtro de estado y paginación. Cada fila ofrece "Editar" y "Dar de baja" o, si la categoría está dada de baja, "Reactivar".
- **Formulario:** nombre, descripción, categoría padre (el selector excluye a la propia categoría y a sus subcategorías, para evitar ciclos en la jerarquía), posición y estado.
- **Operaciones rechazadas:** si la API rechaza una baja (la categoría tiene subcategorías activas o es la única de algún producto) o una reactivación (el nombre lo usa otra categoría o la categoría padre está dada de baja), la pantalla muestra el motivo y, en el caso de la baja, cuántos elementos la impiden.

Módulo admin-users. Resuelve la gestión de usuarios (RF-12) y solo está disponible para el administrador:

- **Listado:** nombre, email, rol, estado (activo, inactivo o dado de baja), obligación de cambiar la contraseña y último acceso, con buscador, filtros de rol y estado, y paginación. Cada fila ofrece "Editar", "Enviar contraseña temporal" y "Dar de baja" o, si el usuario está dado de baja, "Reactivar".

- **Alta:** nombre, email, contraseña, confirmación de la contraseña y rol. La contraseña ingresada es temporal: el usuario debe cambiarla en su primer ingreso.
- **Edición:** nombre, email, rol y estado. La contraseña no se edita en este formulario: se utiliza "Enviar contraseña temporal", que envía por mail una clave nueva y obliga a cambiarla al ingresar.
- **Protecciones de interfaz:** en la fila del propio administrador se deshabilitan la baja, el cambio de rol y la desactivación, con un texto que explica el motivo. La API aplica las mismas reglas de forma independiente (ver 8.4).

9.7 Pantallas y módulos

Cada pantalla se construye combinando componentes de distintos módulos. La Tabla 50 muestra, para cada pantalla, los módulos que intervienen y sus componentes principales, y la Tabla 51 vincula los requerimientos funcionales de la sección 2 con los módulos que los resuelven.

Tabla 50. Pantallas del frontend y módulos que intervienen

Pantalla	Módulos	Componentes principales
Inicio	site-layout, catalog, institutional, cart, whatsapp y seo	Header, HeroSection, ServicesStrip, FeaturedSection, CategoriesSection, NewArrivalsSection, AboutTeaser y Footer
Catálogo	site-layout, catalog, cart y seo	Header, SearchInput, filtro de categorías, grilla de productos, Pagination y Footer
Ficha de producto	site-layout, catalog, cart, whatsapp y seo	Header, breadcrumb, ProductGallery, ProductInfo, RelatedProducts y Footer
Sobre nosotros	site-layout, institutional, whatsapp y seo	Header, AboutPage y Footer
Carrito	cart y whatsapp	CartDrawer
Login, olvido y cambio de contraseña	auth	LoginPage, ForgotPasswordPage y ChangePasswordPage
Listado de productos	admin-shell y admin-products	DataTable, ListToolbar y ConfirmDialog
Alta y edición de producto	admin-shell y admin-products	Formulario de producto e ImageManager
Listado de categorías	admin-shell y admin-categories	DataTable, ListToolbar y ConfirmDialog
Alta y edición de categoría	admin-shell y admin-categories	Formulario de categoría
Listado de usuarios	admin-shell y admin-users	DataTable, ListToolbar y ConfirmDialog
Alta y edición de usuario	admin-shell y admin-users	Formulario de usuario

Tabla 51. Cobertura de los requerimientos funcionales en el frontend

Código	Requerimiento	Módulos	Pantallas o componentes
RF-01	Página de inicio	catalog, institutional y site-layout	Inicio: HeroSection, FeaturedSection, NewArrivalsSection, CategoriesSection y AboutTeaser

Código	Requerimiento	Módulos	Pantallas o componentes
RF-02	Catálogo de productos	catalog	CatalogPage, con el estado de búsqueda y filtros en la URL
RF-03	Ficha de producto	catalog	ProductGallery, ProductInfo y RelatedProducts
RF-04	Carrito y reserva por WhatsApp	cart y whatsapp	CartDrawer y mensaje de pedido
RF-05	Consulta por WhatsApp	catalog y whatsapp	Botón "Consultar por WhatsApp" de ProductInfo
RF-06	Página institucional	institutional	AboutPage
RF-07	Autenticación	auth	LoginPage, ForgotPasswordPage y ChangePasswordPage
RF-08	Control de acceso por roles	auth y admin-shell	Guards de ruta y menú del panel según el rol
RF-09	Gestión de productos	admin-products	Listado y formulario de producto
RF-10	Gestión de imágenes	admin-products	ImageManager
RF-11	Gestión de categorías	admin-categories	Listado y formulario de categoría
RF-12	Gestión de usuarios	admin-users	Listado y formulario de usuario

Los requerimientos no funcionales también se reflejan en el diseño del frontend: la adaptabilidad (RNF-01), en componentes que cambian de disposición según el ancho de la pantalla, como la grilla de productos o las tablas del panel, que pasan a tarjetas en celulares; el rendimiento y el posicionamiento (RNF-02), en el renderizado en el servidor, el loader de imágenes y el módulo seo; y la seguridad (RNF-03), en el access token conservado solo en memoria y en la validación de los formularios, que complementan los controles de la API.

9.8 Estructura de carpetas

La Figura 7 muestra la estructura de carpetas del proyecto. La carpeta app/ contiene únicamente las rutas, organizadas con grupos de rutas de Next.js: (public), para la tienda, cuyas páginas comparten el layout con el header, el footer y el carrito; y (panel), para las pantallas protegidas, que comparten la estructura del panel y los guards. La carpeta modules/ contiene un directorio por módulo, con solo las capas que cada uno necesita: los módulos más simples, como whatsapp o site-layout, incluyen una única capa.



Figura 7. Estructura de carpetas del frontend

9.9 Integración con el backend

El frontend consume exclusivamente la API REST propia, publicada bajo /api/v1 (ver 8.8); no accede a la base de datos ni al almacenamiento de archivos de forma directa. La Tabla 52 resume los endpoints que utiliza cada módulo.

Tabla 52. Endpoints de la API utilizados por cada módulo del frontend

Módulo	Endpoints	Uso
catalog	GET /products, /products/featured, /products/new, /products/:slug y /categories	Inicio, catálogo y ficha de producto, con revalidación cada 60 segundos.
site-layout	GET /categories	Enlaces a las categorías principales en el footer.
seo	GET /products y /categories	Generación del mapa del sitio con los productos y categorías visibles.
cart, whatsapp e institutional	Ninguno	El carrito usa sus datos guardados y se actualiza desde la ficha; los mensajes y los textos se arman en el cliente.
auth	POST /auth/login, /auth/refresh, /auth/logout, /auth/forgot-password y /auth/change-password	Inicio, renovación y cierre de sesión, recuperación y cambio de contraseña.

Módulo	Endpoints	Uso
admin-products	/admin/products y /admin/products/:productId/images	Listado, alta, edición, baja y reactivación de productos, y gestión de sus imágenes.
admin-categories	/admin/categories	Listado, alta, edición, baja y reactivación de categorías.
admin-users	/admin/users	Listado, alta, edición, baja, reactivación y envío de claves temporales.

Además de los endpoints, la integración se apoya en los siguientes acuerdos con el backend:

- **Sesión:** el access token se envía en el encabezado Authorization de cada solicitud del panel, y la cookie del refresh token viaja automáticamente al renovar la sesión. Por la configuración SameSite=Strict de esa cookie, el frontend y la API deben compartir el dominio registrado (ver 8.7).
- **Imágenes:** la base de datos guarda solo las claves de los archivos y la API devuelve las URL públicas de las variantes WebP (ver 7.7); el loader de imágenes del frontend elige la variante de 400, 800 o 1200 px según el ancho requerido.
- **Errores:** el cliente HTTP traduce los códigos de error de la API (ver Tabla 21) a mensajes en español, y los formularios muestran los errores de conflicto (409) y de reglas de negocio (422) junto al campo afectado.
- **Actualización de la tienda:** los cambios realizados en el panel se aplican de inmediato en la API, y la tienda los refleja al vencer el período de revalidación de 60 segundos.

9.10 Decisiones de diseño fundamentadas

La Tabla 53 resume las decisiones de diseño del frontend, la alternativa considerada y el fundamento de cada elección.

Tabla 53. Decisiones de diseño del frontend

Decisión	Alternativa descartada	Fundamento
Organización por módulo funcional, con capas	Organizar el código por tipo de archivo (componentes, hooks y servicios)	Cada funcionalidad reúne sus tipos, reglas, acceso a datos y componentes; replica la regla de dependencias del backend y permite probar el dominio sin React.
Tienda con SSR e ISR, revalidada cada 60 segundos	Generar la tienda en el navegador	Mejor posicionamiento en buscadores y carga más rápida en celulares; un cambio del panel se refleja en la tienda en, como máximo, un minuto.
Panel renderizado en el cliente	Proteger el panel desde el servidor de Next.js	La cookie del refresh token pertenece al dominio de la API y el servidor de Next.js no la recibe; la seguridad real la aplica la API en cada endpoint.
Access token conservado solo en memoria	Guardarlo en localStorage	Un script inyectado en la página no puede leerlo; al recargar, la sesión se recupera con el refresh token.
Una única renovación ante varios errores 401 simultáneos	Renovar la sesión por cada solicitud rechazada	Con un refresh token rotativo, dos renovaciones en paralelo se interpretarían como reutilización y revocarían todas las sesiones del usuario.
Carrito en localStorage, actualizado al visitar la ficha	Endpoint para revalidar todos los productos del carrito	No requiere estado en el servidor ni un endpoint adicional; el aviso en el carrito y la confirmación por WhatsApp cubren los cambios de precio o stock.

Decisión	Alternativa descartada	Fundamento
El carrito no se vacía al enviar el pedido	Vaciarlo automáticamente	El cliente puede no completar el envío en WhatsApp; puede vaciarlo manualmente cuando lo desee.
Loader propio para las imágenes	Optimización de imágenes de Vercel	El backend ya genera las variantes WebP; se evita el cupo limitado de optimizaciones del plan gratuito.
Íconos de categorías resueltos por slug, sin contadores de productos	Guardar el ícono en la base y mostrar la cantidad de productos por categoría	No requiere cambios en el backend ni en el modelo de datos; como el slug es inmutable, la asociación no se pierde al renombrar.
Búsqueda, filtros y página en la URL	Mantenerlos solo en el estado del componente	Los enlaces pueden compartirse, el botón "atrás" funciona y los buscadores indexan las páginas de cada categoría.
SKU de solo lectura y sin subtítulo en el formulario	SKU y subtítulo editables	Coherente con el SKU generado por el sistema e inmutable (ver 7.5); la descripción cumple la función del subtítulo.
Baja y reactivación como acciones del listado	Incluir "dado de baja" como opción del selector de estado	Separa el estado Activo o Inactivo de la baja lógica, en línea con los endpoints DELETE y restore de la API (ver 8.8).

Referencias

Los documentos técnicos propios se encuentran en el repositorio del proyecto (ver Anexo 1.1).

Diagrama entidad-relación · Librería Mi Sueño. Repositorio del Trabajo Final Integrador, GitHub.

<https://github.com/FrancoGhilardi/TFI-TUP-Ghilardi-Chiaravalloti/blob/main/backend/docs/Diagrama-entidad-relacion.md>

Backend · Librería Mi Sueño. Repositorio del Trabajo Final Integrador, GitHub.

<https://github.com/FrancoGhilardi/TFI-TUP-Ghilardi-Chiaravalloti/blob/main/backend/docs/Backend.md>

Frontend · Librería Mi Sueño. Repositorio del Trabajo Final Integrador, GitHub.

<https://github.com/FrancoGhilardi/TFI-TUP-Ghilardi-Chiaravalloti/blob/main/frontend/docs/Frontend.md>

OWASP Foundation. (s.f.). *Password Storage Cheat Sheet*. OWASP Cheat Sheet Series. Recuperado el 26 de septiembre de 2026, de

https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

Resend. (s.f.). *Pricing*. Recuperado el 26 de septiembre de 2026, de <https://resend.com/pricing>

Supabase. (s.f.). *Pricing & fees*. Recuperado el 26 de septiembre de 2026, de <https://supabase.com/pricing>

Anexos

1.1 Repositorio de GitHub

- Repositorio del Trabajo Final Integrador, GitHub.
<https://github.com/FrancoGhilardi/TFI-TUP-Ghilardi-Chiaravalloti>

1.2 Diagramas técnicos del proyecto

Los diagramas completos del proyecto se encuentran en el repositorio de GitHub (Anexo 1.1), en formato Markdown con diagramas Mermaid que GitHub muestra directamente como imágenes:

- **Diagrama entidad-relación:** diagrama entidad-relación, descripción de las tablas y script de creación (DDL) de la base de datos.
<https://github.com/FrancoGhilardi/TFI-TUP-Ghilardi-Chiaravalloti/blob/main/backend/docs/Diagrama-entidad-relacion.md>
- **Backend:** diagramas de clases del backend por módulo y por capa, con sus reglas de negocio, flujos, endpoints y decisiones de diseño.
<https://github.com/FrancoGhilardi/TFI-TUP-Ghilardi-Chiaravalloti/blob/main/backend/docs/Backend.md>
- **Frontend:** módulos, rutas y pantallas del frontend, con su estructura de carpetas, la integración con la API y las decisiones de diseño.
<https://github.com/FrancoGhilardi/TFI-TUP-Ghilardi-Chiaravalloti/blob/main/frontend/docs/Frontend.md>